

FIT5217

Natural Language Processing

Week 3 - Sequence Labelling

Trang Vu

Slides build on various sources: Jurafsky and Manning, Nigel Collier

Overview

1. **Word categories**
2. Part-of-Speech (POS) Tagging
3. Other Sequence Labelling Problems
4. Hidden Markov Model (HMM)
5. HMM: Observation Likelihood
6. HMM: Most Likely State Sequence
7. HMM: Supervised Learning
8. Evaluation

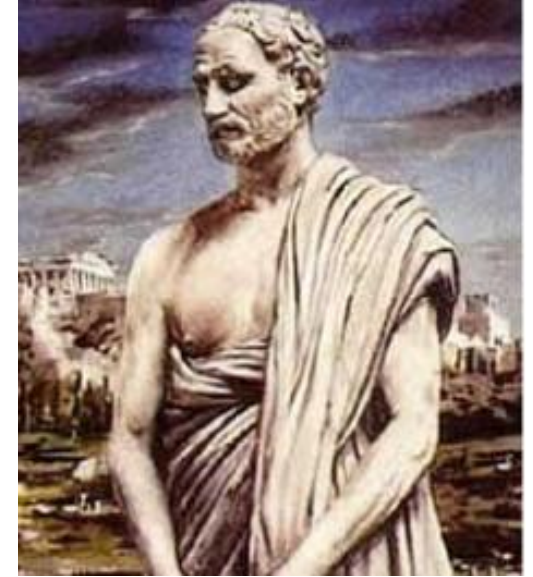
Word Categories

- Dionysius Thrax wrote (c. 100BCE) about eight parts-of-speech as a way of classifying words based on their functions:

nouns verbs articles adverbs preposition
conjunction pronoun participle

- Thrax's list and minor variations of it dominated European language grammars and dictionaries for 2000 years:

nouns verbs **adjectives** adverbs preposition
conjunction pronoun **interjection**



Word Categories

- **Open class words (or content words)**
 - nouns, main verbs, adjectives, adverbs, numbers, ...
 - Mostly content-bearing: they refer to objects, actions and properties in the world
 - There is no limit to what these words are, new ones are added all the time
- **Closed class words (or function words)**
 - pronouns, determiners, prepositions, connectives, modal verbs, interjections, ...
 - There are a limited number of these

Open class ("content") words

Nouns

Proper

Janet
Italy

Common

cat, cats
mango

Verbs

Main

eat
went

Adjectives

old green tasty

Adverbs

slowly yesterday

Numbers

122,312
one

Interjections *Ow hello*

... more

Closed class ("function")

Determiners *the some*

Conjunctions *and or*

Pronouns *they its*

Auxiliary

can
had

Prepositions *to with*

Particles *off up*

... more

Overview

1. Word categories
- 2. Part-of-Speech (POS) Tagging**
3. Other Sequence Labelling Problems
4. Hidden Markov Model (HMM)
5. HMM: Observation Likelihood
6. HMM: Most Likely State Sequence
7. HMM: Supervised Learning
8. Evaluation

English POS Tags – Different Conventions

- Brown corpus used a set of 87 POS tags.
- The British National has 61 tags.
- Most common in NLP today is the Penn Treebank



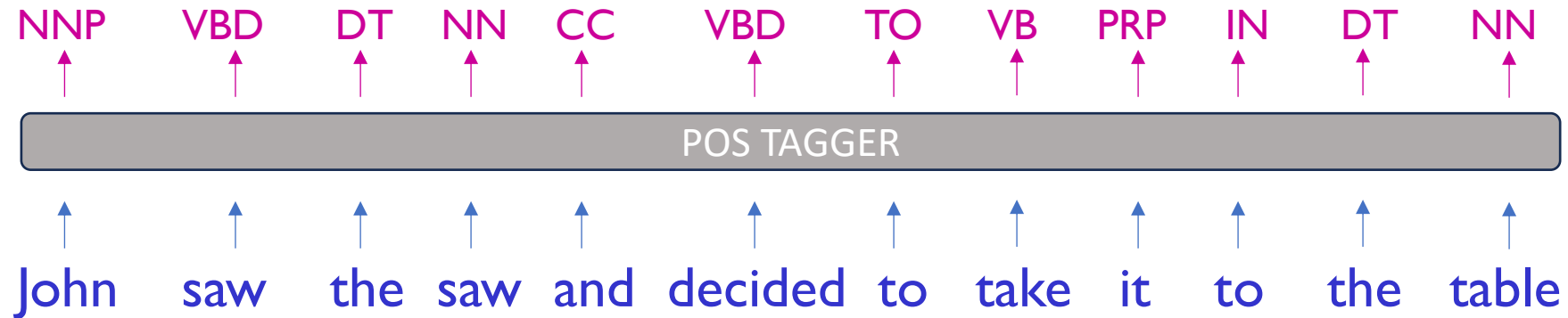
1.	CC	Coordinating conjunction
2.	CD	Cardinal number
3.	DT	Determiner
4.	EX	Existential <i>there</i>
5.	FW	Foreign word
6.	IN	Preposition or subordinating conjunction
7.	JJ	Adjective
8.	JJR	Adjective, comparative
9.	JJS	Adjective, superlative
10.	LS	List item marker
11.	MD	Modal
12.	NN	Noun, singular or mass
13.	NNS	Noun, plural
14.	NNP	Proper noun, singular
15.	NNPS	Proper noun, plural
16.	PDT	Predeterminer
17.	POS	Possessive ending
18.	PRP	Personal pronoun
19.	PRPS	Possessive pronoun
20.	RB	Adverb
21.	RBR	Adverb, comparative
22.	RBS	Adverb, superlative
23.	RP	Particle
24.	SYM	Symbol
25.	TO	<i>to</i>
26.	UH	Interjection
27.	VB	Verb, base form
28.	VBD	Verb, past tense
29.	VBG	Verb, gerund or present participle
30.	VBN	Verb, past participle
31.	VBP	Verb, non-3rd person singular present
32.	VBZ	Verb, 3rd person singular present
33.	WDT	Wh-determiner
34.	WP	Wh-pronoun
35.	WP\$	Possessive wh-pronoun
36.	WRB	Wh-adverb

Check this for Penn Treebank POS tags: https://www.ling.upenn.edu/courses/Fall_2003/ling001/penn_treebank_pos.html

English POS Tags – Penn Treebank

- Noun (person, place or thing)
 - Singular (NN): dog, fork
 - Plural (NNS): dogs, forks
 - Proper (NNP, NNPS): John, Springfields
 - Personal pronoun (PRP): I, you, he, she, it
 - Wh-pronoun (WPP): who, what
- Verb (actions and processes)
 - Base, infinitive (VB): eat
 - Past tense (VBD): ate
 - Gerund (VBG): eating
 - Past participle (VBN): eaten
 - Non 3rd person singular present tense (VBP): eat
 - 3rd person singular present tense: (VBZ): eats
 - Modal (MD): should, can
 - To (TO): to (to eat)
- Adjective (modify nouns)
 - Basic (JJ): red, tall
 - Comparative (JJR): redder, taller
 - Superlative (JJS): reddest, tallest
- Adverb (modify verbs)
 - Basic (RB): quickly
 - Comparative (RBR): quicker
 - Superlative (RBS): quickest
- Preposition (IN): on, in, by, to, with
- Determiner:
 - Basic (DT) a, an, the
 - WH-determiner (WDT): which, that
- Coordinating Conjunction (CC): and, but, or
- Particle (RP): off (took off), up (put up)

Part-of-Speech Tagging



- Annotate each word in a sentence with a POS
- Lowest level of syntactic analysis (i.e., used to be done to produce parse trees)

Ambiguity in POS Tagging

Words often have more than one POS – “back”:

- The back door = JJ (Adjective)
- On my back = NN (Noun, Singular or Mass, Uncountable)
- Win the voters back = RB (Adverb)
- Promised to back the bill = VB (Verb Base form)

Ambiguity in POS Tagging - Continued

- Based on Brown corpus: 11.5% of word **types** and 40% of word **tokens** are ambiguous.
- Baseline is already 90%
 - Baseline is performance of the simplest possible method: Tag each word with its most frequent tag, and tag unknown words as nouns
- Partly because many words are not ambiguous
- Current state-of-the-art is about 98% (checkout the [leaderboard](#))

Uses for POS Tagging

- Text-to-speech (how to pronounce “lead”, i.e. is it a verb or a noun?)
- If we POS tagged a collection of text we could do some basic IR by writing regex. E.g., to extract all noun phrases/compounds:
 - The car
 - A white horse
 - The red big box
 - A round red big cup
 - ...
- As input to syntactic parsers
- If you know the tags, you can back off from words to their POS tag categories (i.e., another way of smoothing in language modelling)

Question: Can you write a regex to catch all potential noun phrases in a corpus?

Uses for POS Tagging

- Sentiment Analysis: "The movie was incredibly amazing."
- POS Tags:
 - incredibly → RB (Adverb)
 - amazing → JJ (Adjective) → Strong positive sentiment
- By focusing on adjectives and adverbs, sentiment analysis models can weigh sentiment more accurately.

Uses for POS Tagging

Named Entity Recognition:

Sentence

"Elon Musk founded Tesla in 2003."

"Google's headquarters is in Mountain View, California."

"The iPhone 15 was released in September 2023."

"Microsoft acquired LinkedIn for \$26 billion."

"The Eiffel Tower is one of the most visited landmarks in Paris."

Detected Named Entities

Elon Musk (**PERSON**), Tesla (**ORG**), 2003 (**DATE**)

Google (**ORG**), Mountain View (**GPE**), California (**GPE**)

iPhone 15 (**PRODUCT**), September 2023 (**DATE**)

Microsoft (**ORG**), LinkedIn (**ORG**), \$26 billion (**MONEY**)

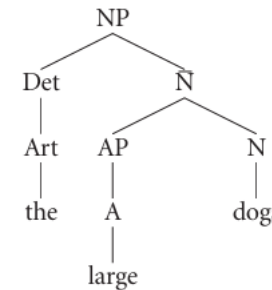
Eiffel Tower (**FAC**), Paris (**GPE**)

POS Tagging Approaches

- **Rule-Based:** Human crafted rules based on lexical and other linguistic knowledge. Expensive!

Table 7.1. Expansions of NP

NP →	N	<i>dogs</i>
	Det N	<i>the dogs</i>
	Det A N	<i>the large dogs</i>
	Det AP N	<i>the loudly barking dogs</i>
	Det N PP	<i>the dog in the yard</i>
	Det A N PP	<i>the ferocious dog behind the fence</i>
	Det AP N PP	<i>the wildly yapping dog on the sofa</i>
	Pro	<i>He</i>
	PN	<i>Goldy</i>



- **Learning-Based:** Trained on human annotated corpora like the Penn Treebank.
 - **Statistical models:** Hidden Markov Model (HMM), Conditional Random Field (CRF), ...
 - **Neural networks:** Recurrent networks like Long Short Term Memory (LSTMs)

[Brill, Eric. "A simple rule-based part of speech tagger." *Speech and Natural Language: Proceedings of a Workshop Held at Harriman, New York, February 23-26, 1992*. 1992.](#)

[Brinton, Laurel J. *The structure of modern English: A linguistic introduction*. Vol. 1. John Benjamins Publishing, 2000.](#)

Side Note: Classification

- Typical machine learning addresses the problem of classifying a feature-vector description into a fixed number of classes. There are many standard learning methods for this task:
 - Decision Trees
 - Naïve Bayes
 - Logistic Regression / Maximum Entropy (MaxEnt)
 - Perceptron and Neural Networks
 - Support Vector Machines (SVMs)
- Naïve Solution: Treat POS tagging as predicting a POS label for words **independently**. Question: what could go wrong? (recall our example for the word “back”)

But ...

- Standard classification problem assumes individual cases are disconnected and independent (i.i.d. - independently and identically distributed).
- Many NLP problems do not satisfy this assumption and involve making many connected decisions which are mutually dependent.
- More sophisticated learning and inference techniques are needed to handle such situations in general.

Overview

1. Word categories
2. Part-of-Speech (POS) Tagging
- 3. Other Sequence Labelling Problems**
4. Hidden Markov Model (HMM)
5. HMM: Observation Likelihood
6. HMM: Most Likely State Sequence
7. HMM: Supervised Learning
8. Evaluation

Sequence Labeling Problem

- Many NLP problems can be viewed as sequence labeling.
- Each token in a sequence is assigned a label.
- Labels of tokens are **dependent** on the labels of other tokens in the sequence, particularly their neighbors (**not i.i.d**).

Example: Information Extraction

- Identify phrases in language that refer to specific types of entities and relations in text.
- Named Entity Recognition (NER) is the task of identifying names of people, places, organizations, ..., in text.

people

organizations

places

- **Michael Dell** is the CEO of **Dell Computer Corporation** and lives in **Austin Texas**.
- Extract pieces of information relevant to a specific application, e.g. used car ads:
 - year make model mileage price
 - For sale, **2002 Toyota Prius**, **20,000 mi**, **\$15K** or best offer.

Example: Semantic Role Labeling

- For each clause, determines the semantic role played by each noun phrase that is an argument to the **verb**.

agent patient source destination instrument
John drove Mary from Austin to Dallas in his Toyota Prius.

instrument patient
The hammer broke the window.

Probabilistic Sequence Models

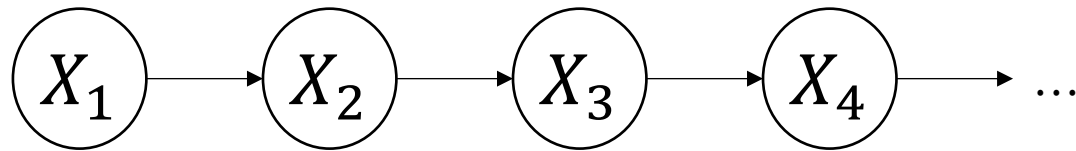
- Probabilistic sequence models allow integrating uncertainty over **multiple interdependent classifications** and **collectively determining** the most likely global assignment.
- Two standard models
 - Hidden Markov Model (HMM)
 - Conditional Random Field (CRF)

Overview

1. Word categories
2. Part-of-Speech (POS) Tagging
3. Other Sequence Labelling Problems
- 4. Hidden Markov Model (HMM)**
5. HMM: Observation Likelihood
6. HMM: Most Likely State Sequence
7. HMM: Supervised Learning
8. Evaluation

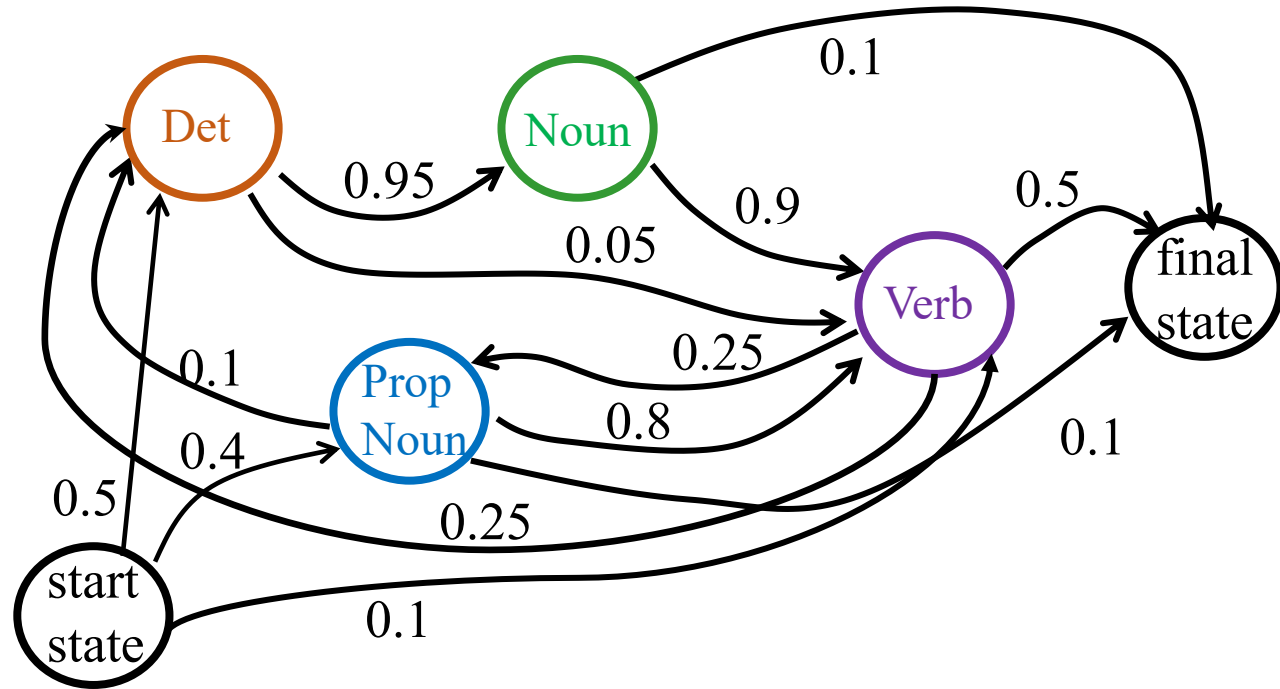
Markov Model / Markov Chain

- Makes Markov assumption that next state only depends on the current state and independent of previous history. We used this in n-gram language modelling!
- Markov Chain:



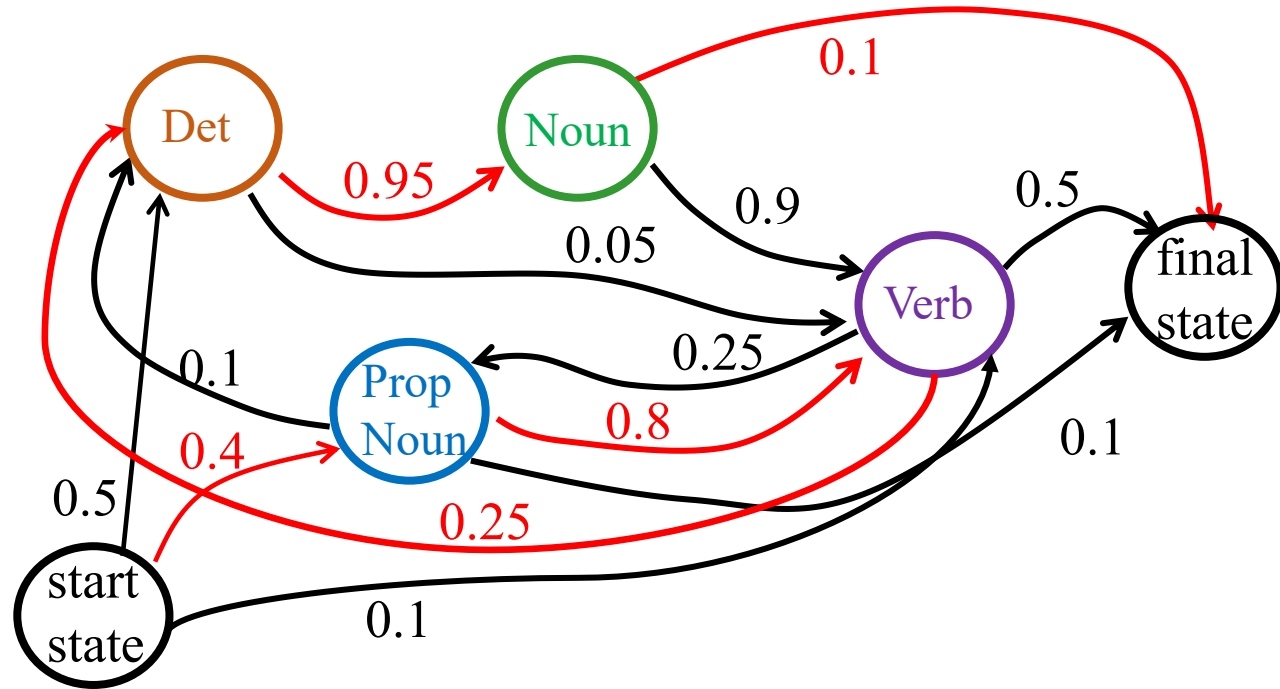
A finite state machine with probabilistic state transitions

- Assume this finite state machine :
- It is probabilistic: each connection (i.e., transition) is assigned a probability , the sum of the edges exiting each node is 1.
- This is a complete directed graph, and missing edges here indicate 0 probability.



A finite state machine with probabilistic state transitions

- Assume this finite state machine :
- It is probabilistic: each connection (i.e., transition) is assigned a probability , the sum of the edges exiting each node is 1.
- This is a complete directed graph, and missing edges here indicate 0 probability.

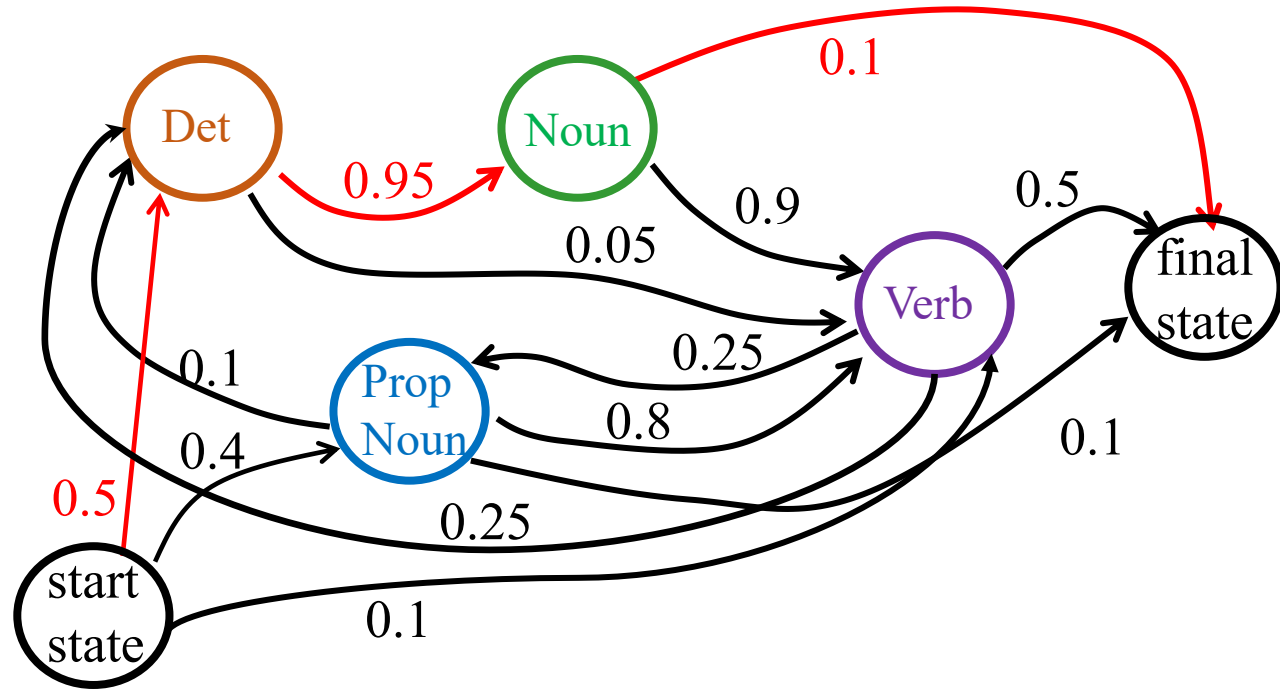


Given a sequence, simply multiply the values of the corresponding edges to compute its probability, by applying chain rule!

$$P(\text{PropNoun Verb Det Noun}) = P(\text{PropNoun} \mid \text{start}) * P(\text{Verb} \mid \text{PropNoun}) \\ P(\text{Det} \mid \text{Verb}) * P(\text{Noun} \mid \text{Det}) * P(\text{Noun} \mid \text{stop}) = 0.4 * 0.8 * 0.25 * 0.95 * 0.1$$

A finite state machine with probabilistic state transitions

- Assume this finite state machine :
- It is probabilistic: each connection (i.e., transition) is assigned a probability , the sum of the edges exiting each node is 1.
- This is a complete directed graph, and missing edges here indicate 0 probability.

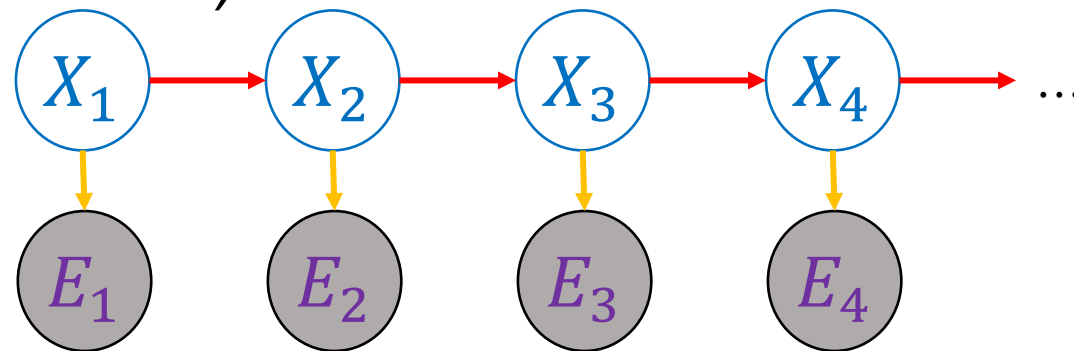


Given a sequence, simply multiply the values of the corresponding edges to compute its probability, by applying chain rule!

$$P(\text{Det Noun}) = 0.5 * 0.95 * 0.1$$

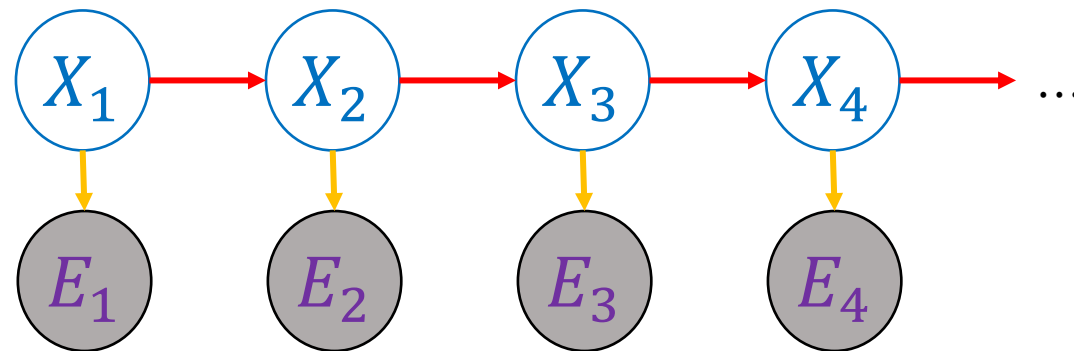
Hidden Markov Model (HMM)

- Probabilistic generative model for sequences.
- Assumes an underlying set of **hidden** (unobserved, latent) states in which the model can be (e.g., parts of speech).
- Assumes probabilistic **transitions** between states over time (e.g., transition from one POS to another POS as sequence is generated).
- Assumes a *probabilistic* **generation** of **tokens** from states (e.g. words generated for each POS).

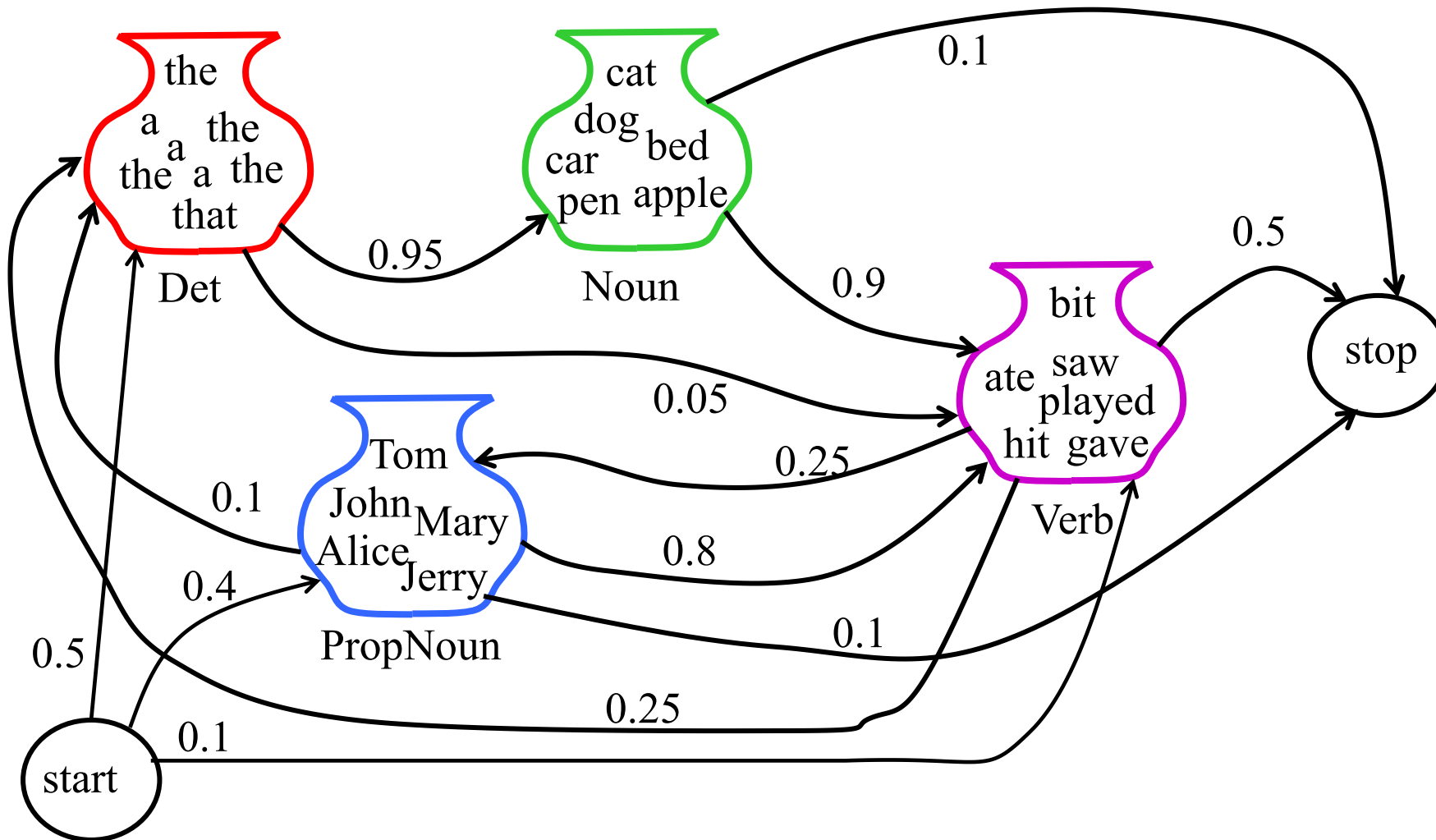


Hidden Markov Model (HMM)

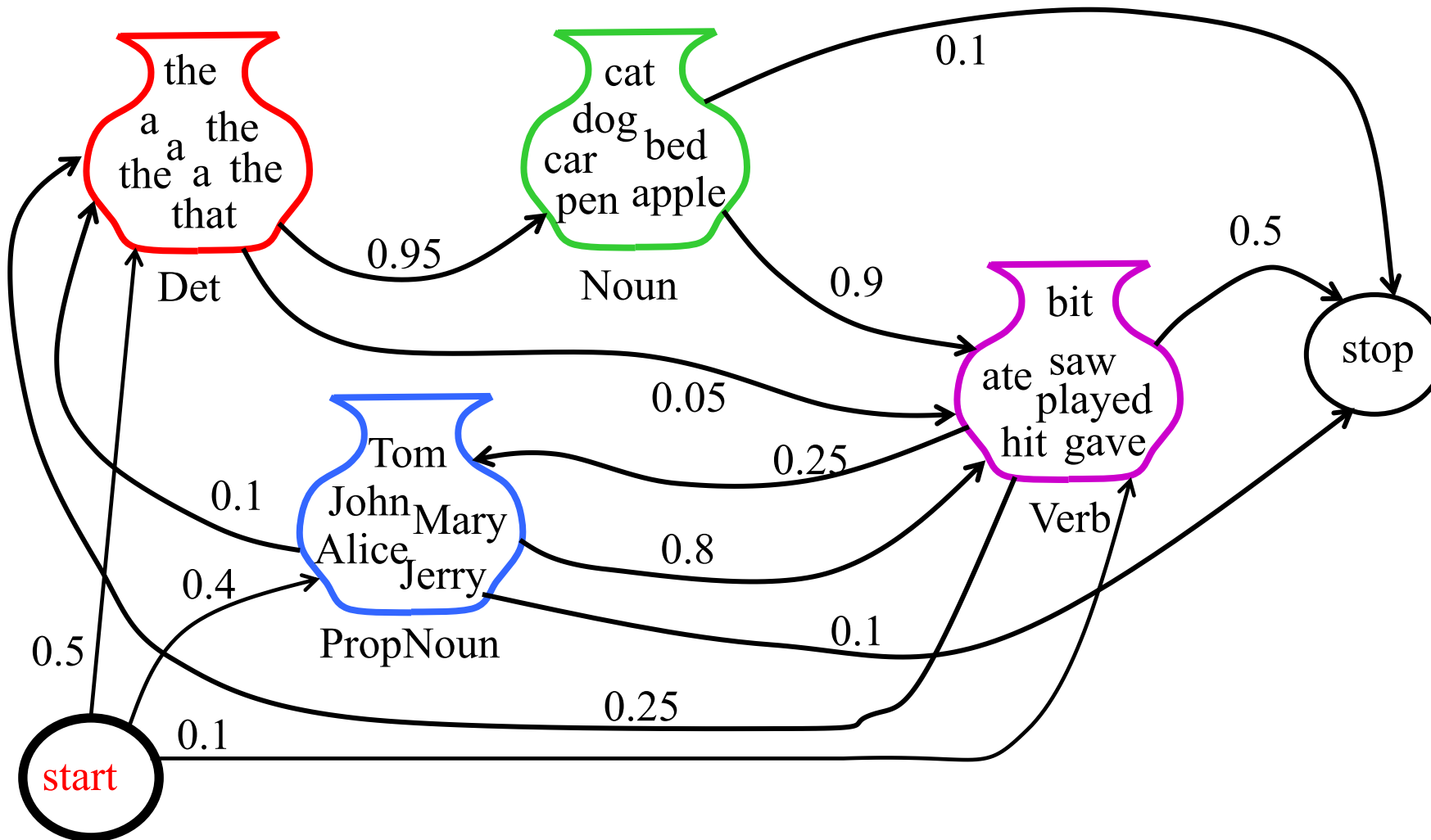
A Markov chain is useful when we need to compute a probability for a sequence of observable events. In many cases, however, the *events we are interested in are hidden*: we don't observe them directly. *For example we don't normally observe part-of-speech tags in a text*. Rather, we see words, and must infer the tags from the word sequence. We call the *tags hidden because they are not observed*.



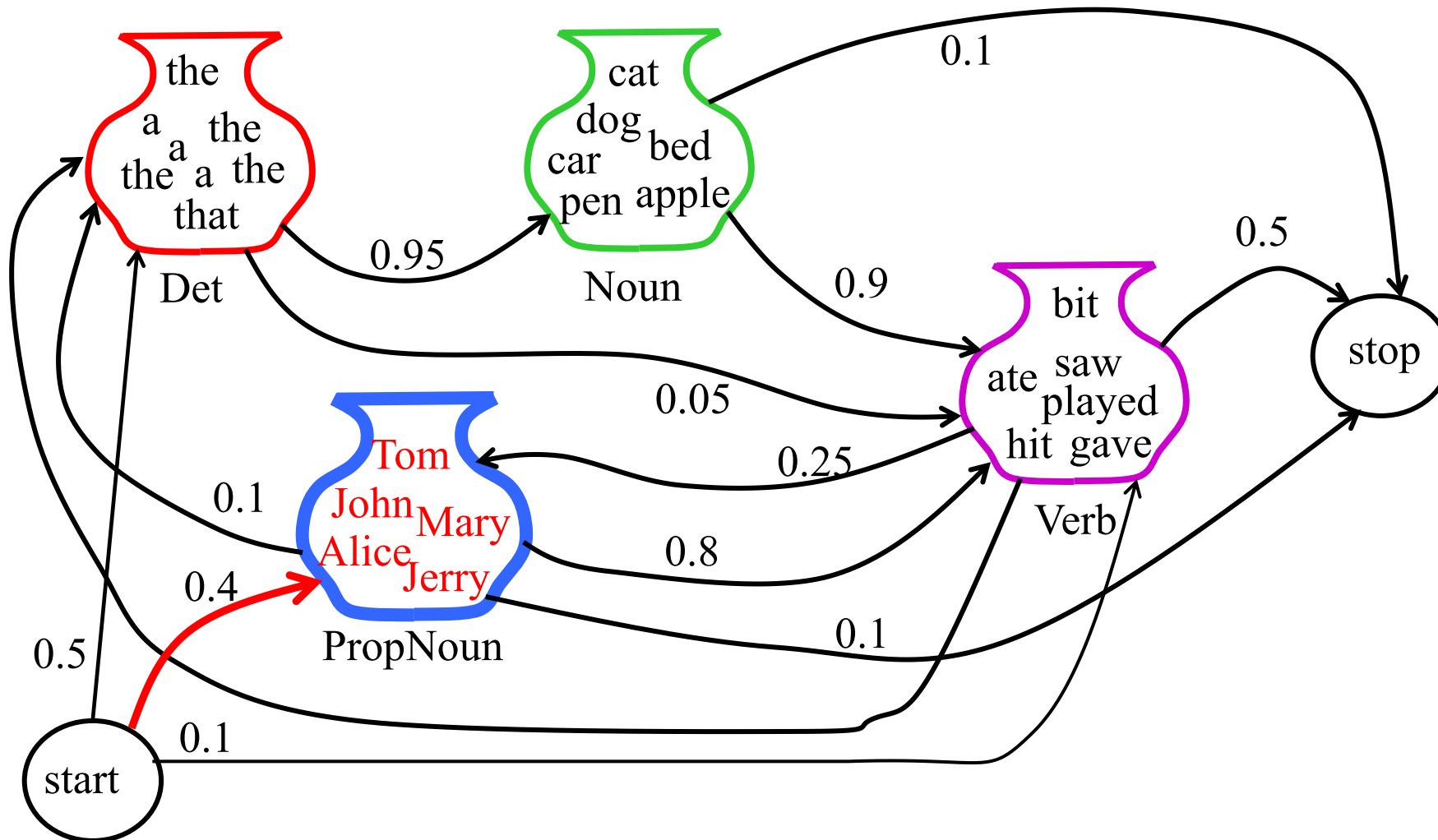
Sample HMM for POS



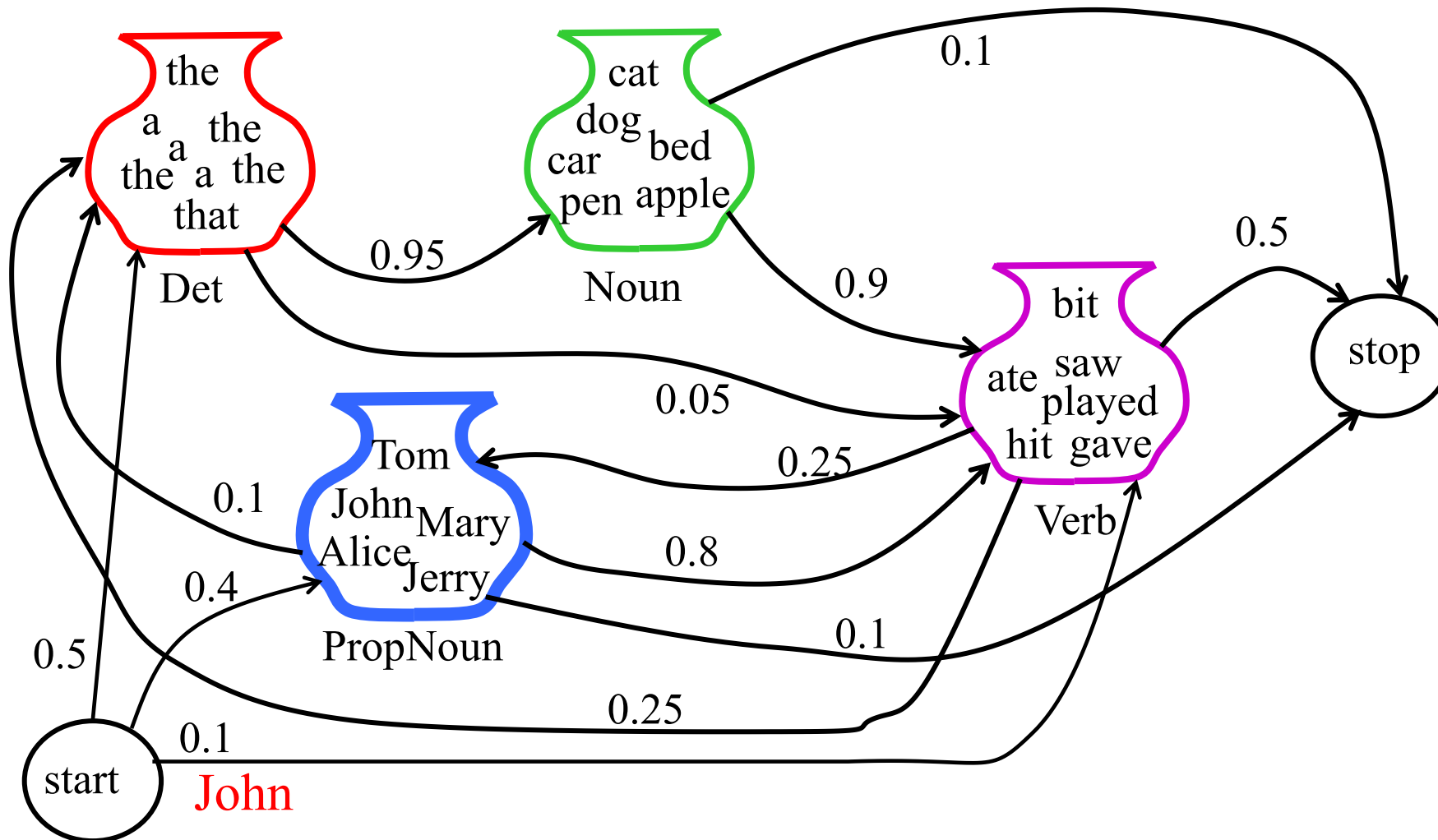
Sample HMM for Generation



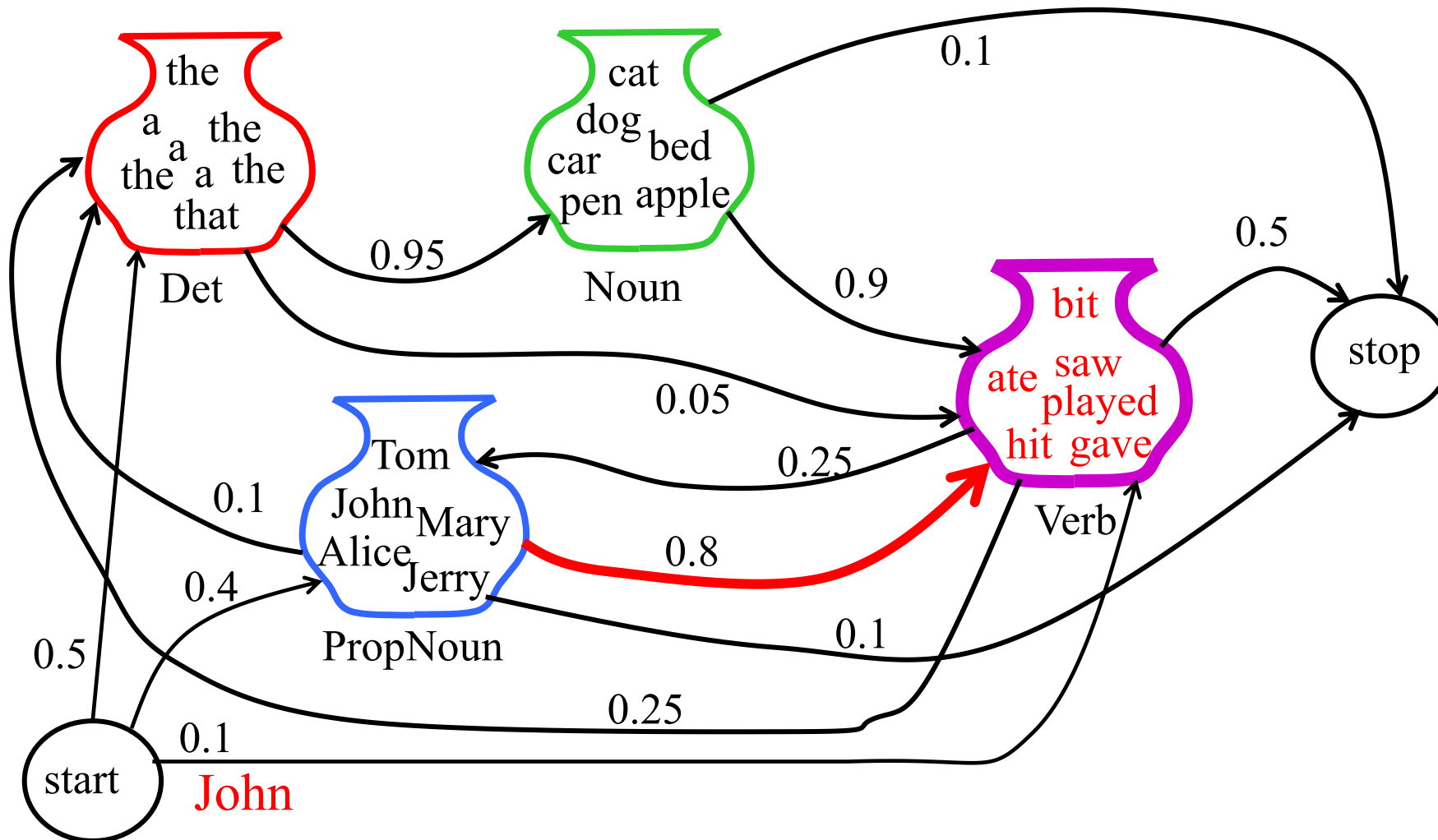
Sample HMM for Generation



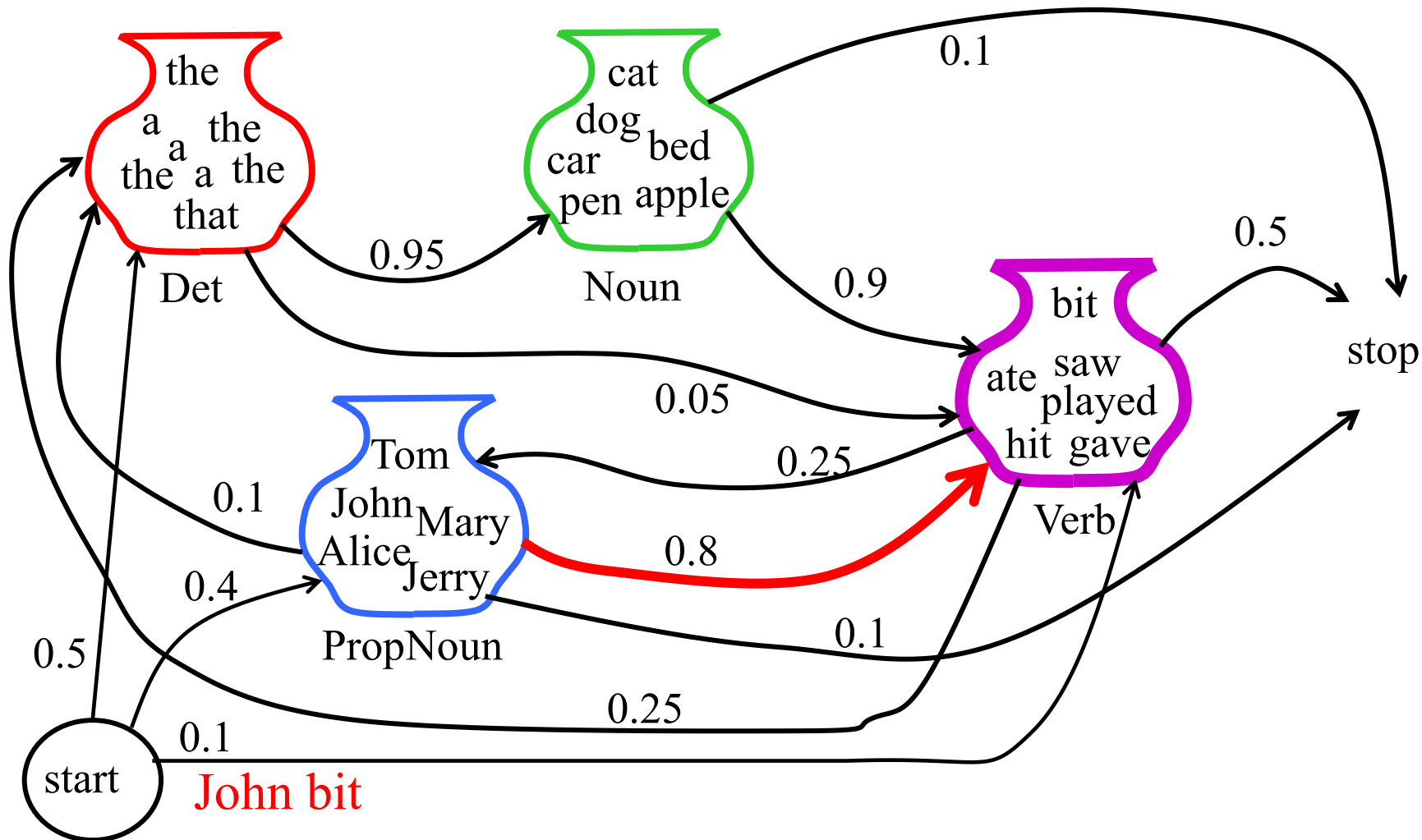
Sample HMM for Generation



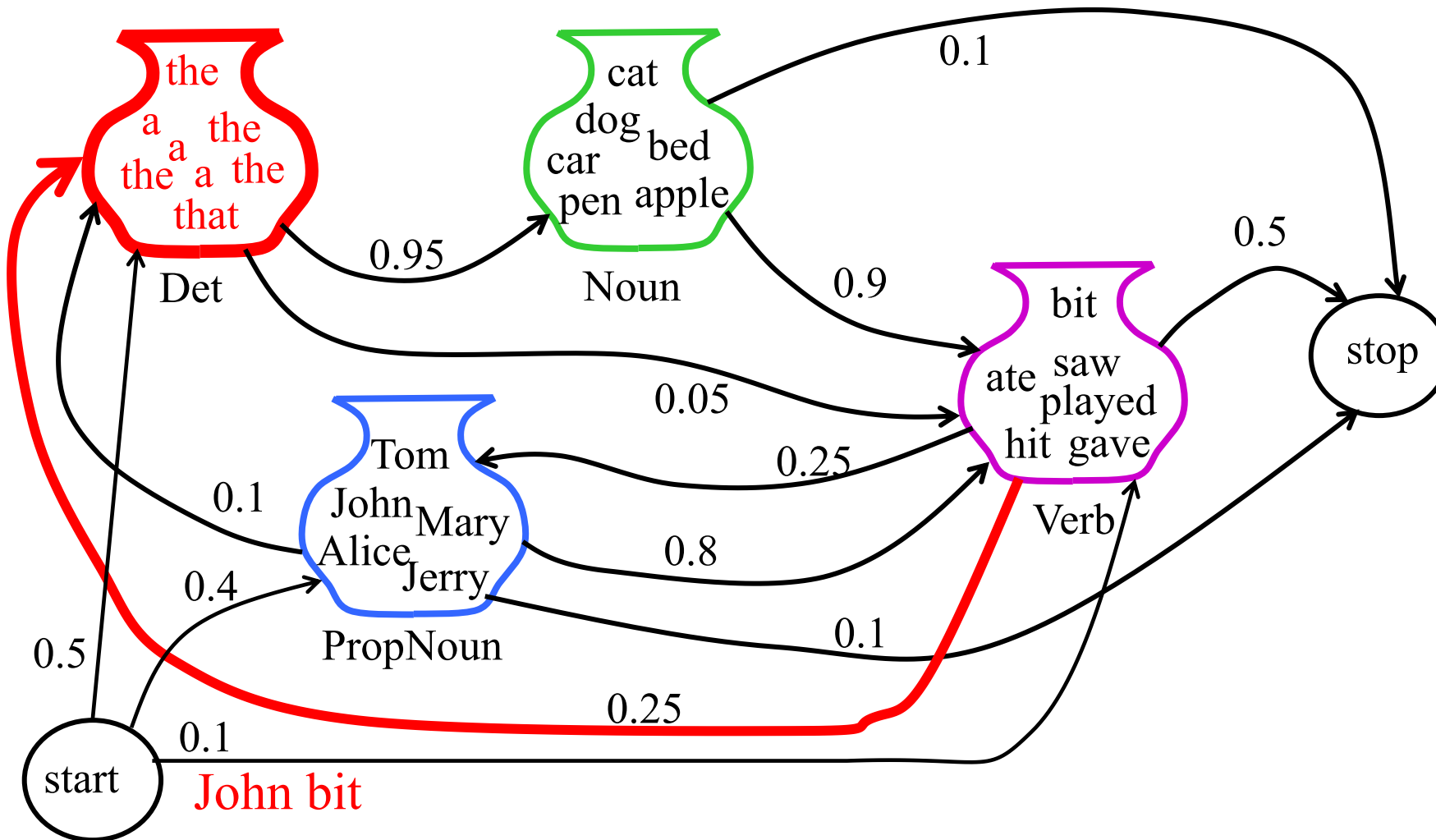
Sample HMM for Generation



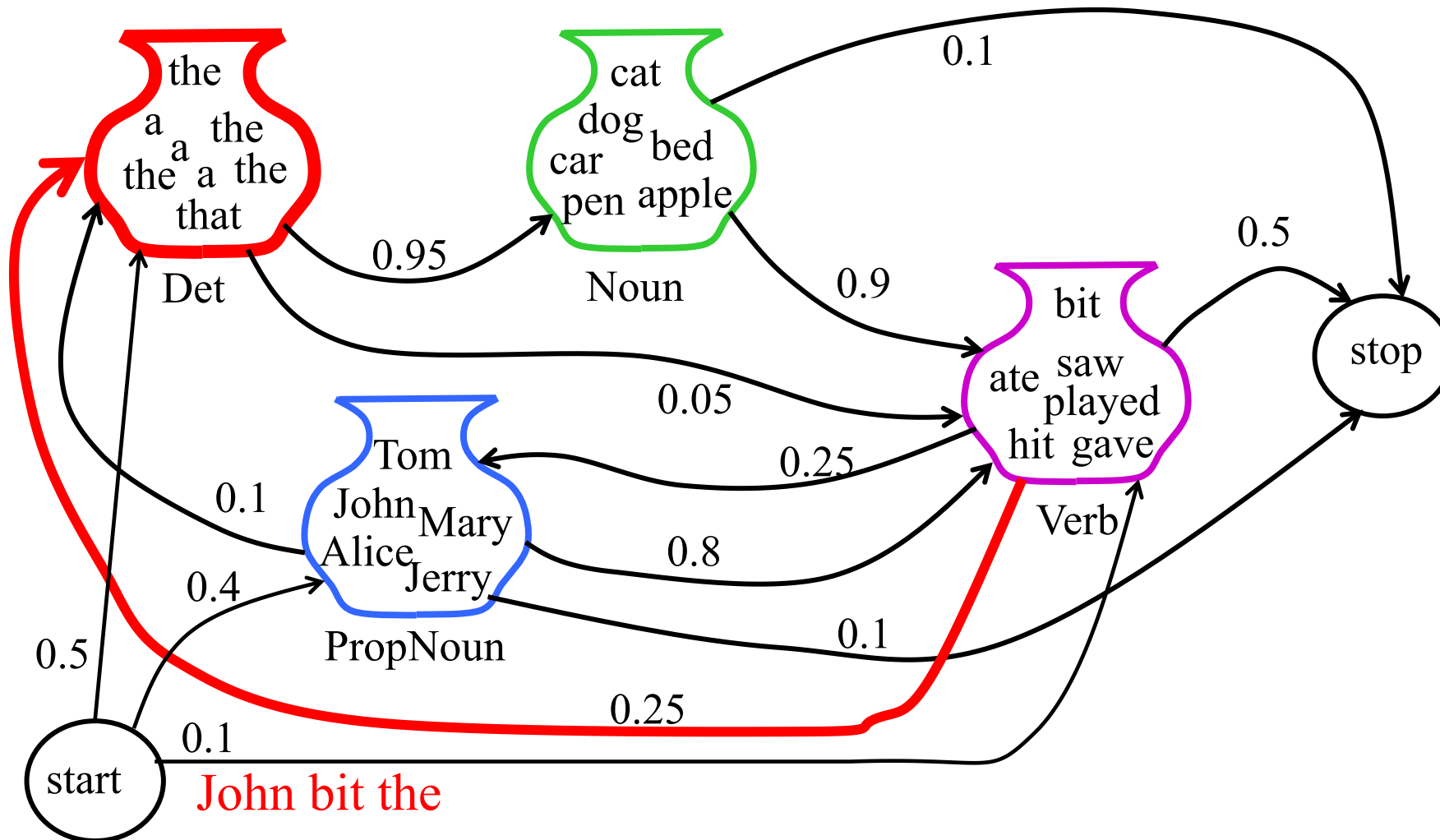
Sample HMM for Generation



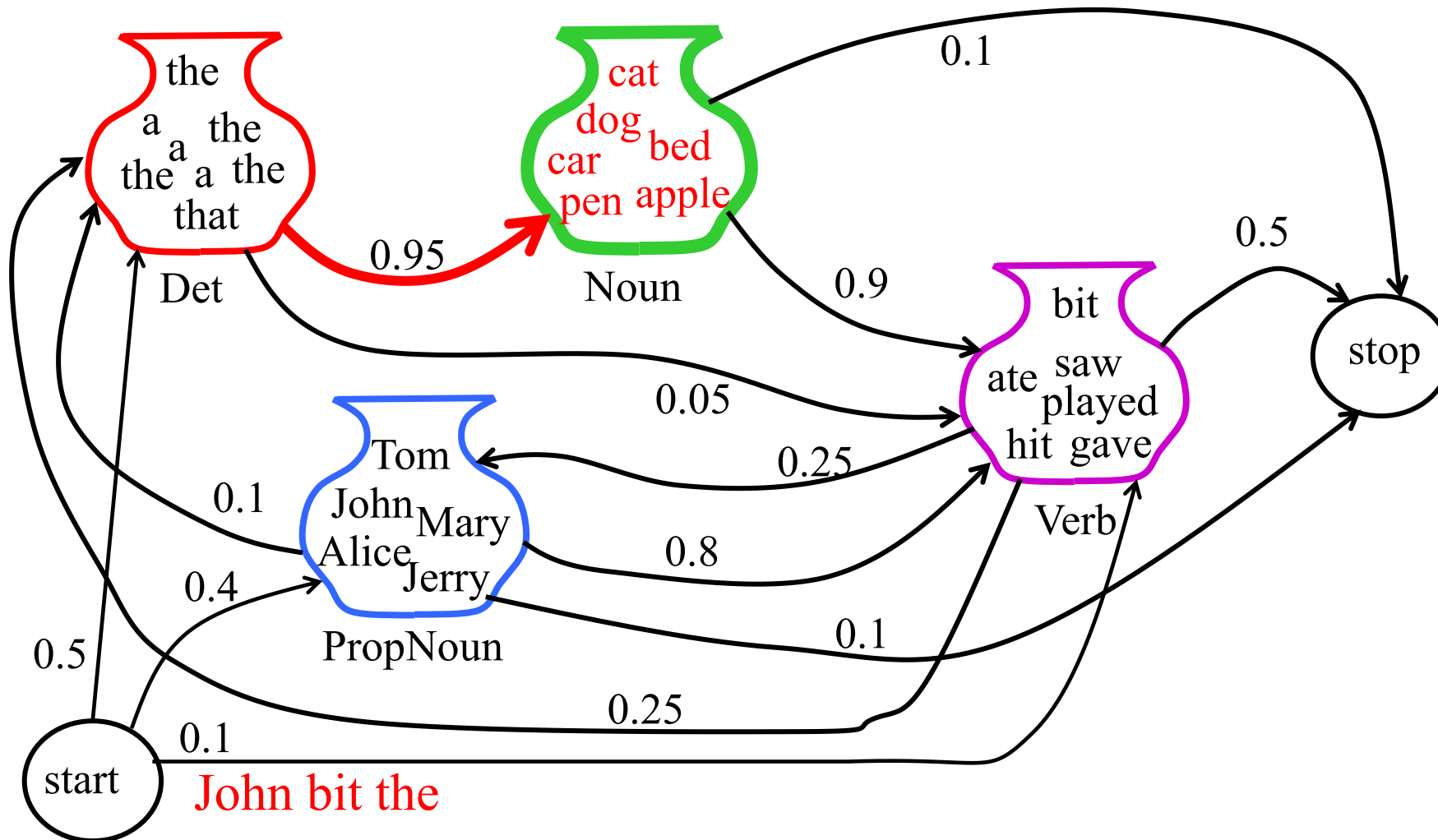
Sample HMM for Generation



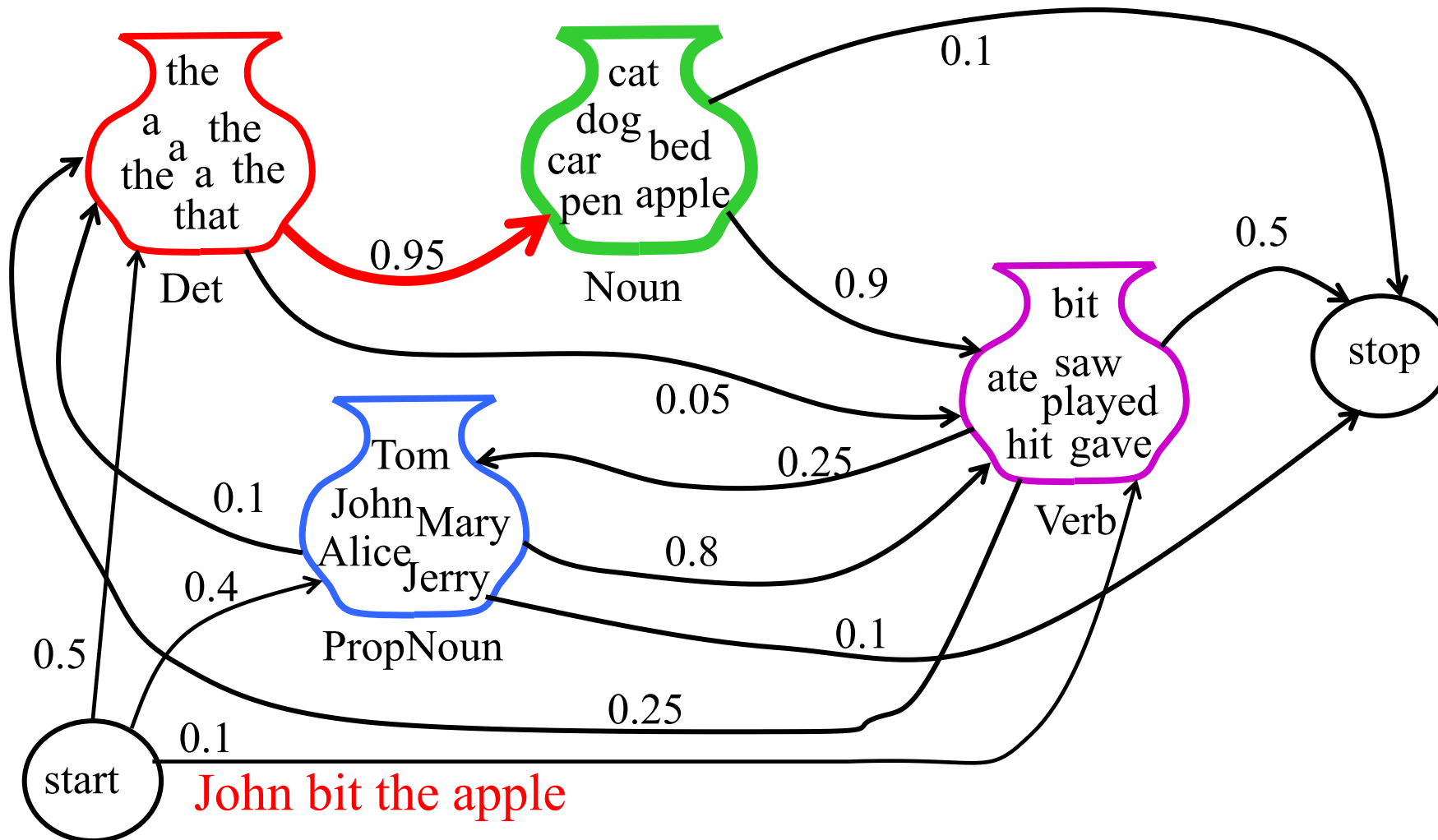
Sample HMM for Generation



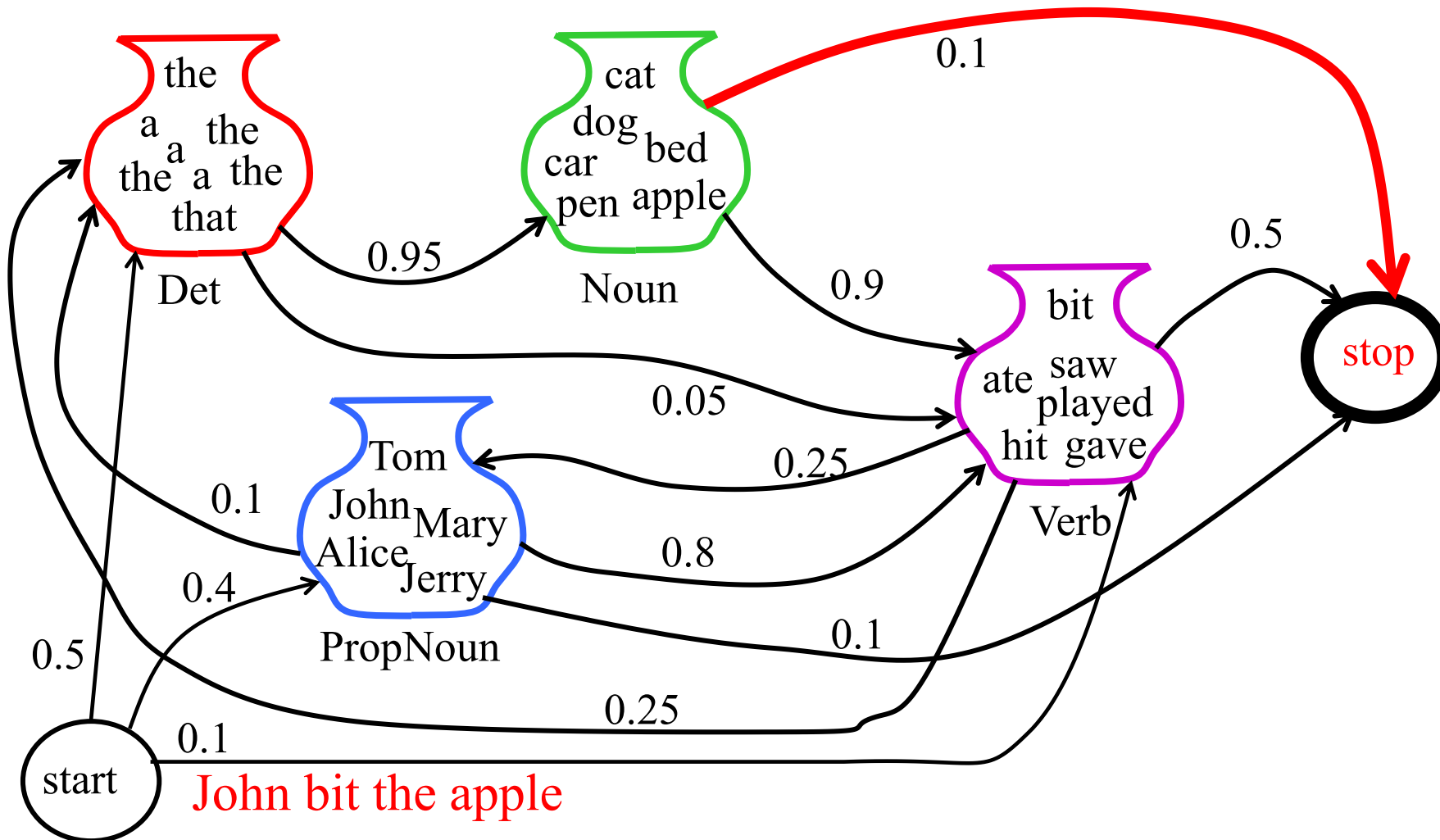
Sample HMM for Generation



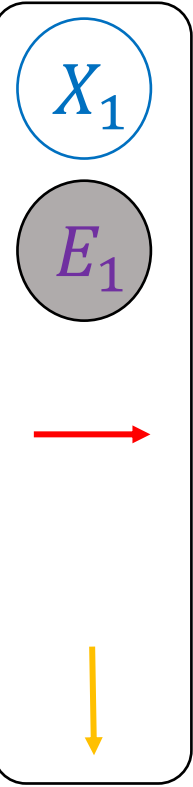
Sample HMM for Generation



Sample HMM for Generation



Formal Definition of HMM (for POS Tagging)



- A set of $N + 2$ states $S = \{s_0, s_1, s_2, \dots, s_N, s_F\}$, where N is the number of POS tags
- A set of M possible observations $V = \{v_1, v_2, \dots, v_M\}$, where M is the size of the vocabulary
- A state transition (i.e., edge) probability distribution $A = \{a_{ij}\}$,

$$a_{ij} = P(q_{t+1} = s_j \mid q_t = s_i) \quad 1 \leq i, j \leq N \text{ and } i = 0, j = F$$

- Observation probability distribution (i.e., probability of a word given a POS tag) for each state $B = \{b_j(k)\}$,

$$b_j(k) = P(v_k \text{ at } t \mid q_t = s_j) \quad 1 \leq j \leq N \quad 1 \leq k \leq M$$

- We denote the total parameter set $\lambda = \{A, B\}$

A first-order HMM

With a (commonly made) first-order Markov chain, the probability of a particular state depends only on the previous state:

$$P(q_t = s_j | q_1 q_2 \dots q_{t-1}) = P(q_t = s_j | q_{t-1})$$

The probability of an output observation o_t depends only on the state that produced the observation q_t and not on any other states or any other observations:

$$P(o_t | q_1 q_2 \dots, o_1, o_2 \dots) = P(o_t | q_t)$$

Three Useful HMM Tasks

- Observation Likelihood: To classify and order sequences.
- Most likely state sequence (Decoding): To tag each token in a sequence with a label.
- Maximum likelihood training (Learning): To train models to fit empirical training data.

Problem 1 (Likelihood):	Given an HMM $\lambda = (A, B)$ and an observation sequence O , determine the likelihood $P(O \lambda)$.
Problem 2 (Decoding):	Given an observation sequence O and an HMM $\lambda = (A, B)$, discover the best hidden state sequence Q .
Problem 3 (Learning):	Given an observation sequence O and the set of states in the HMM, learn the HMM parameters A and B .

Overview

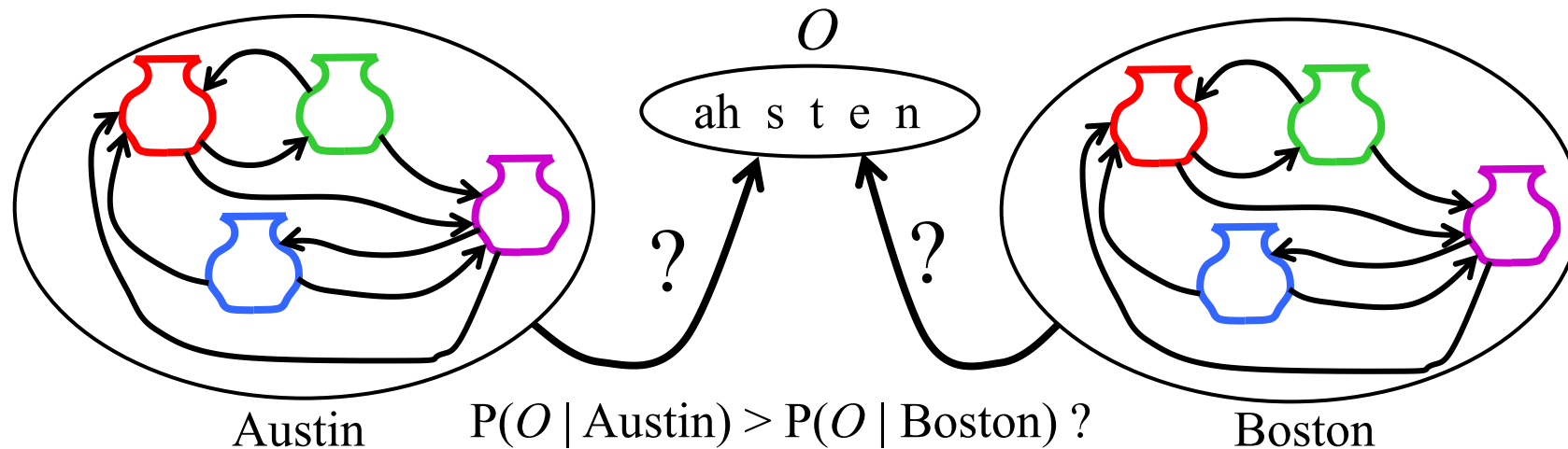
1. Word categories
2. Part-of-Speech (POS) Tagging
3. Other Sequence Labelling Problems
4. Hidden Markov Model (HMM)
- 5. HMM : Observation Likelihood**
6. HMM: Most Likely State Sequence
7. HMM: Supervised Learning
8. Evaluation

HMM: Observation Likelihood

- Given a sequence of observations, O , and a model with a set of parameters, λ (i.e., probability matrices of HMM), what is the probability that this observation was generated by this model: $P(O | \lambda) = ?$
- Use HMM like a **language model**
- Useful for two tasks
 - Sequence classification
 - Most likely sequence

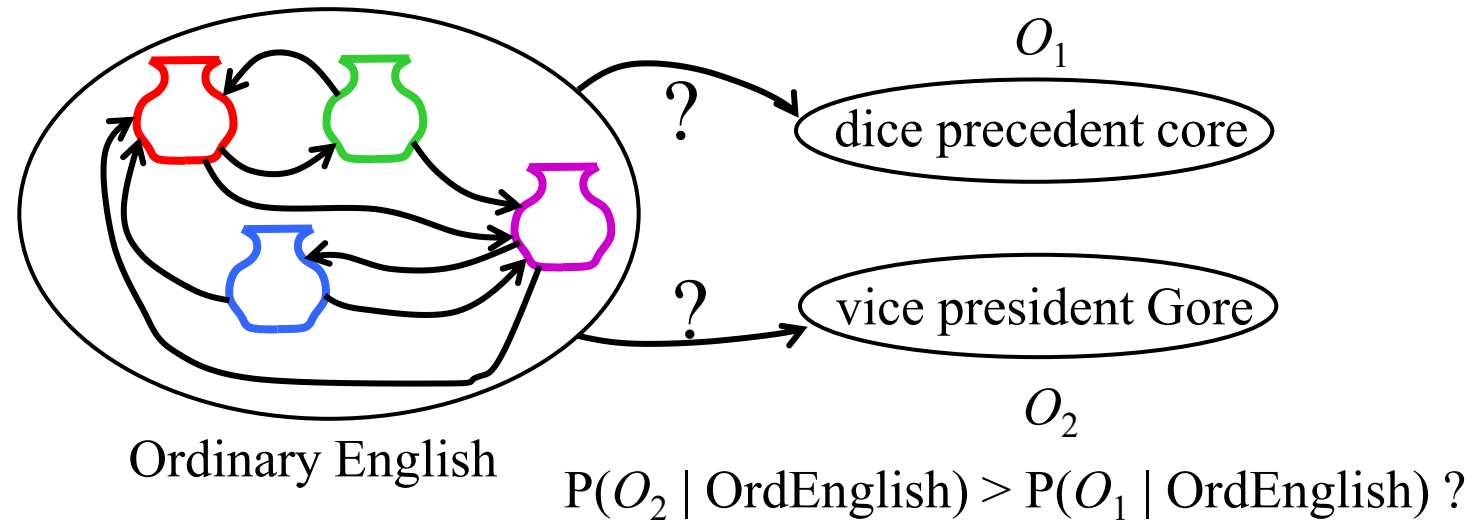
Sequence Classification

- Assume an HMM is available for each category (i.e. language).
- What is the most likely category for a given observation sequence, i.e. which category's HMM is most likely to have generated it?
- Used in speech recognition to find most likely word model to have generate a given sound or phoneme sequence.



Most Likely Sequence

- Of two or more possible sequences, which one was most likely generated by a given model?
- Used to score alternative word sequence interpretations in speech recognition.



Observation Likelihood - Naïve Solution

- In the naïve solution, you calculate the probability of all possible sequence of tags for the given observation and sum them up.
- Let's suppose that we have N possible tag classes and a sentence of length T .
- How many possible tag sequences are there? Let's take $N=17$ and $T=1,2,4,8$. The answer is a lot!

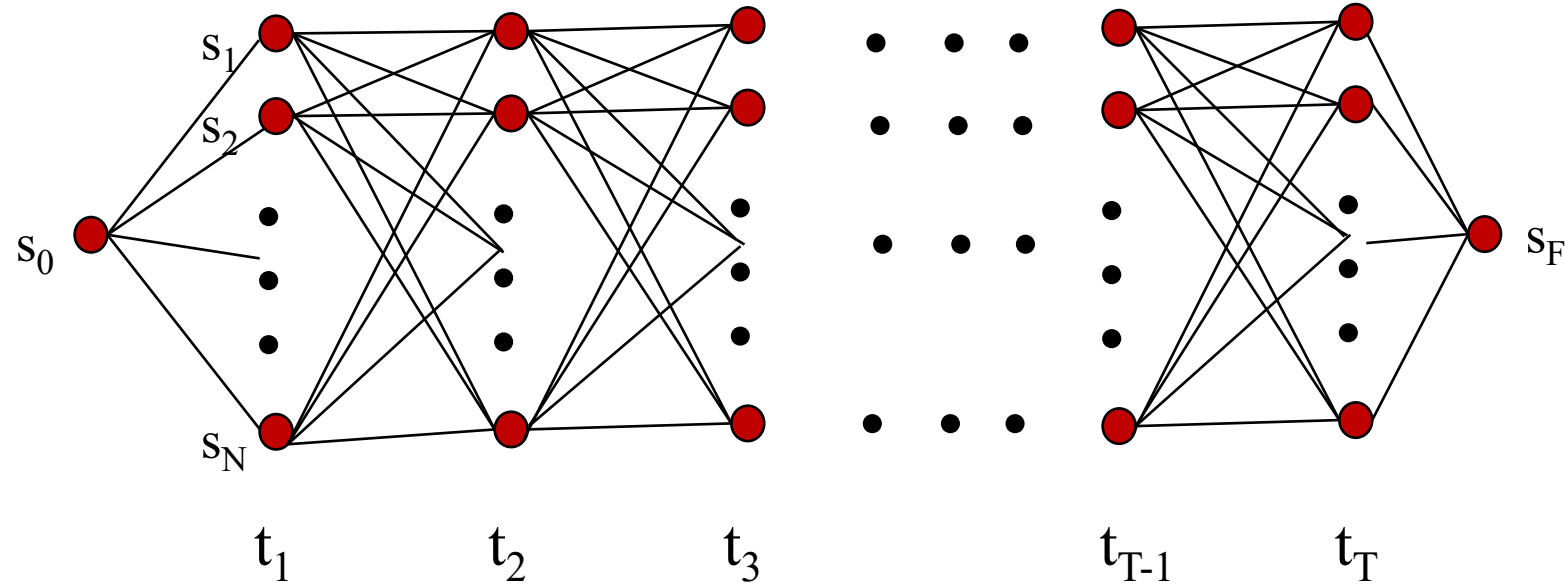
<i>Sentence length (T)</i>	<i>Possible Sequence of POS tags (i.e., 17^T)</i>
1	17
2	289
4	83,421
8	6,975,757,441

- The explosion in the search space!

Observation Likelihood - Efficient Solution

- Intuition: the best path of length t ending in a particular tag must include the best path of length $t-1$ ending in the previous tag. This is due to the Markov assumption.
- **Forward Algorithm**: Uses dynamic programming to exploit this fact to efficiently compute observation likelihood.
 - Compute a ***forward trellis*** that compactly and implicitly encodes information about all possible state paths.

Forward Trellis



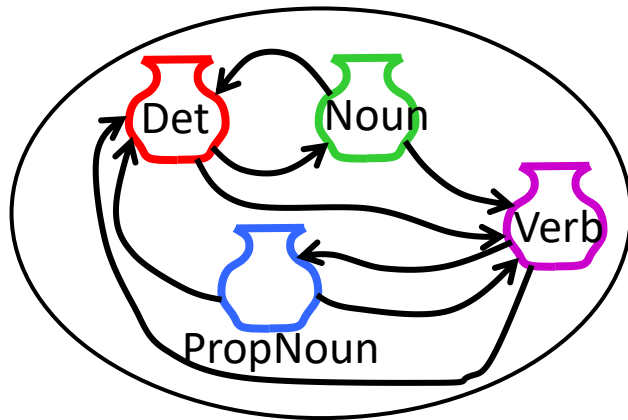
- Continue forward in time until reaching final time point and **sum the forward probabilities** of ending in final state.

Forward Probabilities

- Let $\alpha_t(j)$ be the probability of being in state j after seeing the first t observations (by summing over all initial paths leading to j).

$$\alpha_t(j) = P(o_1, o_2, \dots, o_t, q_t = s_j \mid \lambda)$$

Think of o_t as the t -th word (i.e., “the”) of the sentence you are tagging and think of $q_t = s_j$ as assigning a POS tag s_j (i.e., “DT”) to “the”.

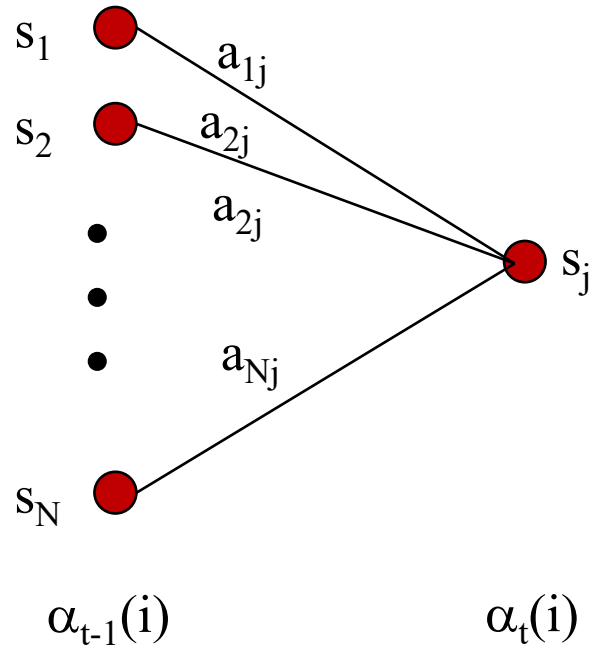


$S = \{\text{start, Det, Noun, PropNoun, Verb, stop}\}$

$O = \text{John bit the apple}$

$$\alpha_2(3) = P(\text{John, bit}, q_2 = \text{PropNoun} \mid \lambda)$$

Forward Step



- Consider all possible ways of getting to s_j at time t by coming from all possible states s_i and determine probability of each.
- Sum these to get the total probability of being in state s_j at time t while accounting for the first $t - 1$ observations.
- Then multiply by the probability of actually observing o_t in s_j .

Computing the Forward Probabilities

- Initialization

$$\alpha_1(j) = a_{0j} b_j(o_1) \quad 1 \leq j \leq N$$

- Recursion

$$\alpha_t(j) = \left[\sum_{i=1}^N \alpha_{t-1}(i) a_{ij} \right] b_j(o_t) \quad 1 \leq j \leq N, \quad 1 < t \leq T$$

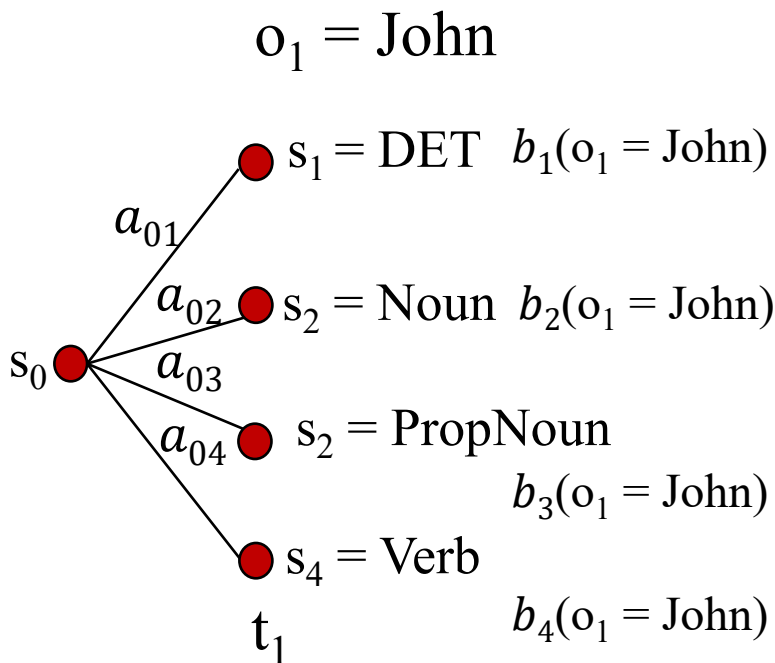
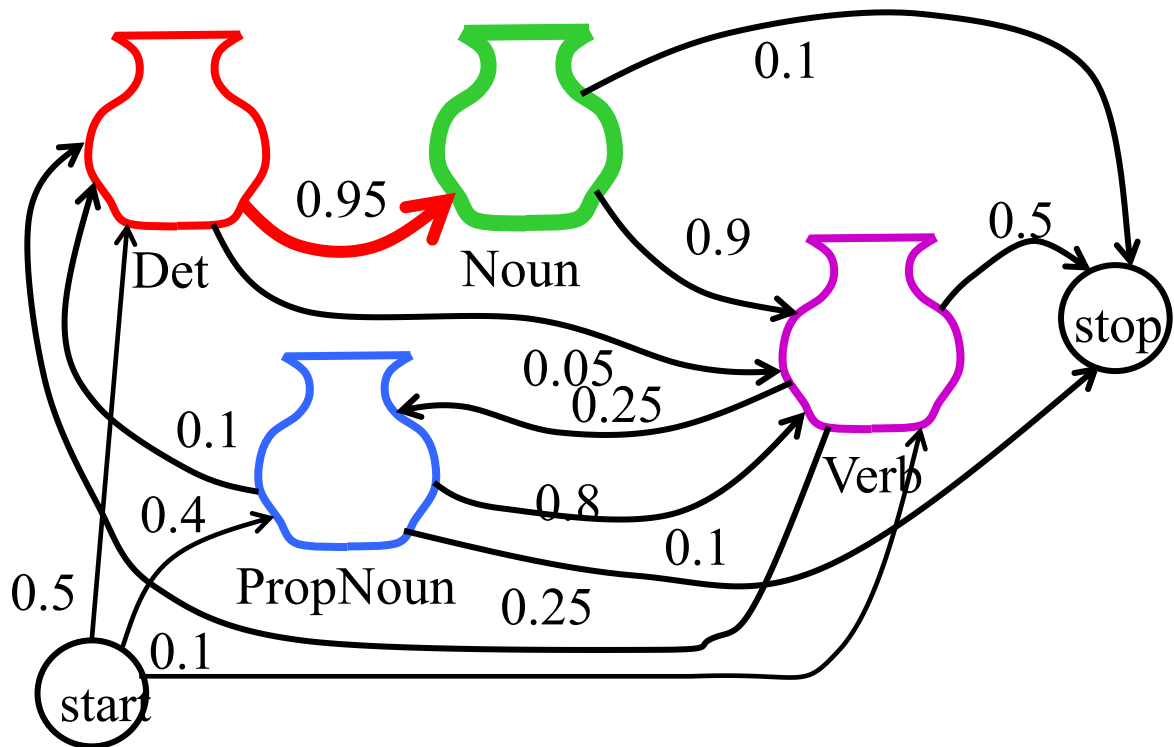
- Termination

$$P(O \mid \lambda) = \alpha_{T+1}(s_F) = \sum_{i=1}^N \alpha_T(i) a_{iF}$$

Computing the Forward Probabilities

- Initialization

$$\alpha_1(j) = a_{0j} b_j(o_1) \quad 1 \leq j \leq N$$



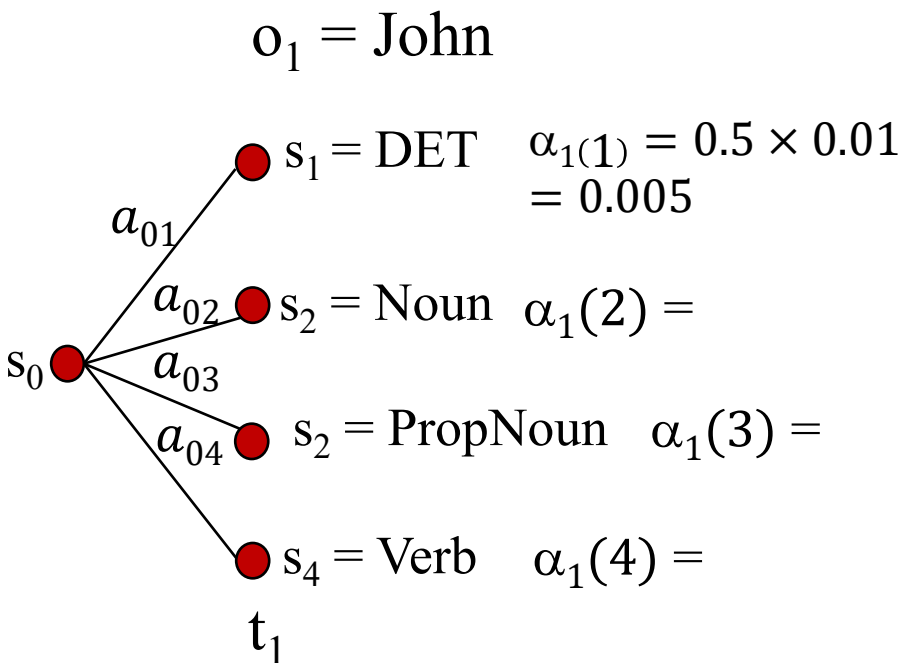
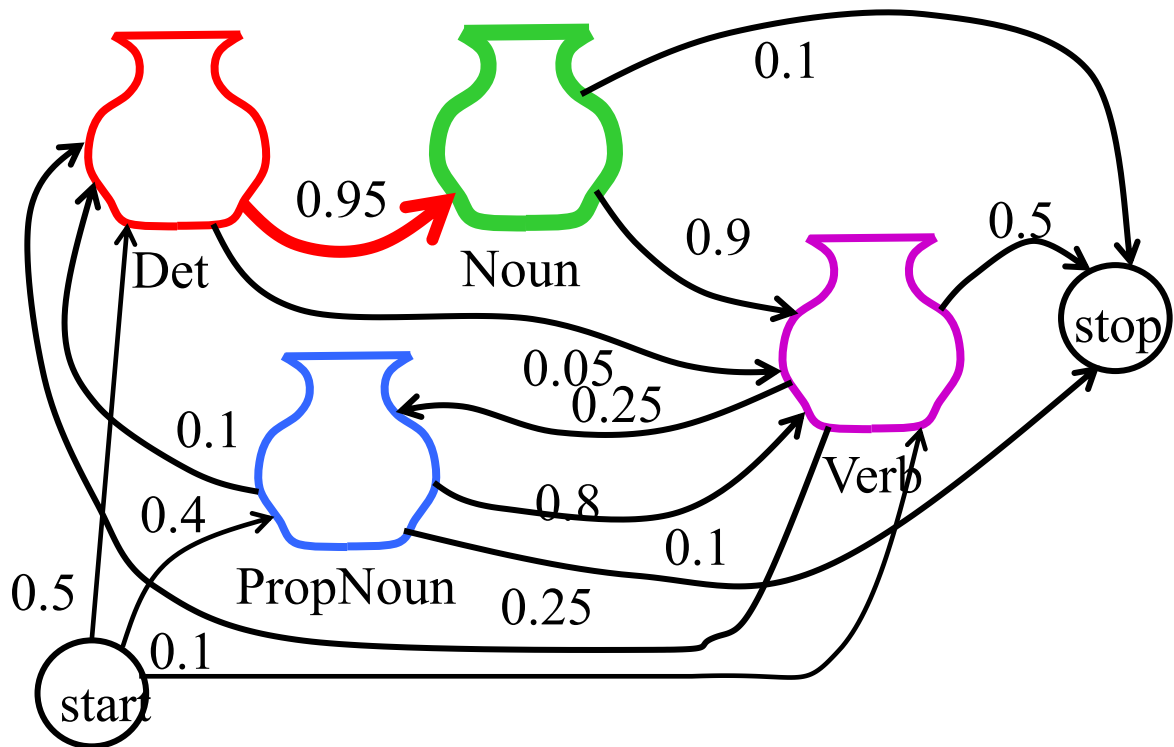
John bit the apple

	John	bit	the	apple
DET	0.01	0.04	0.95	0
Noun	0.2	0.05	0.05	0.7
PropNoun	0.75	0.05	0	0.2
Verb	0	0.9	0.05	0.05

Computing the Forward Probabilities

- Initialization

$$\alpha_1(j) = a_{0j} b_j(o_1) \quad 1 \leq j \leq N$$



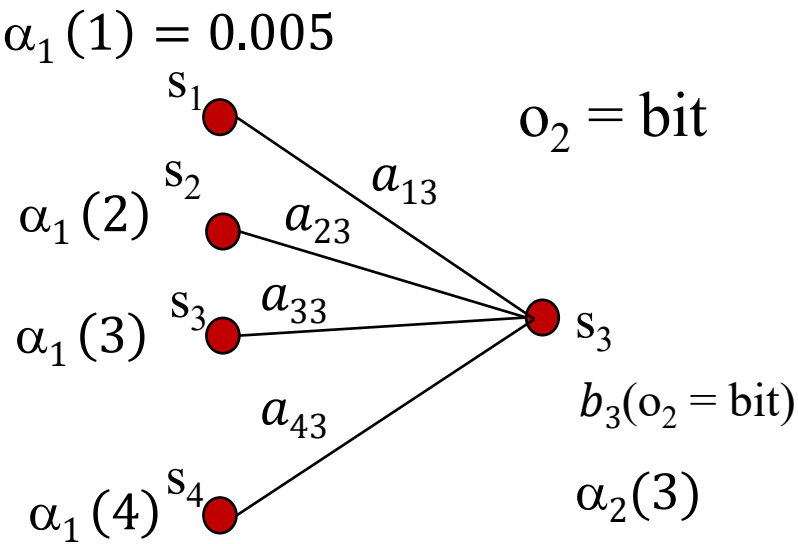
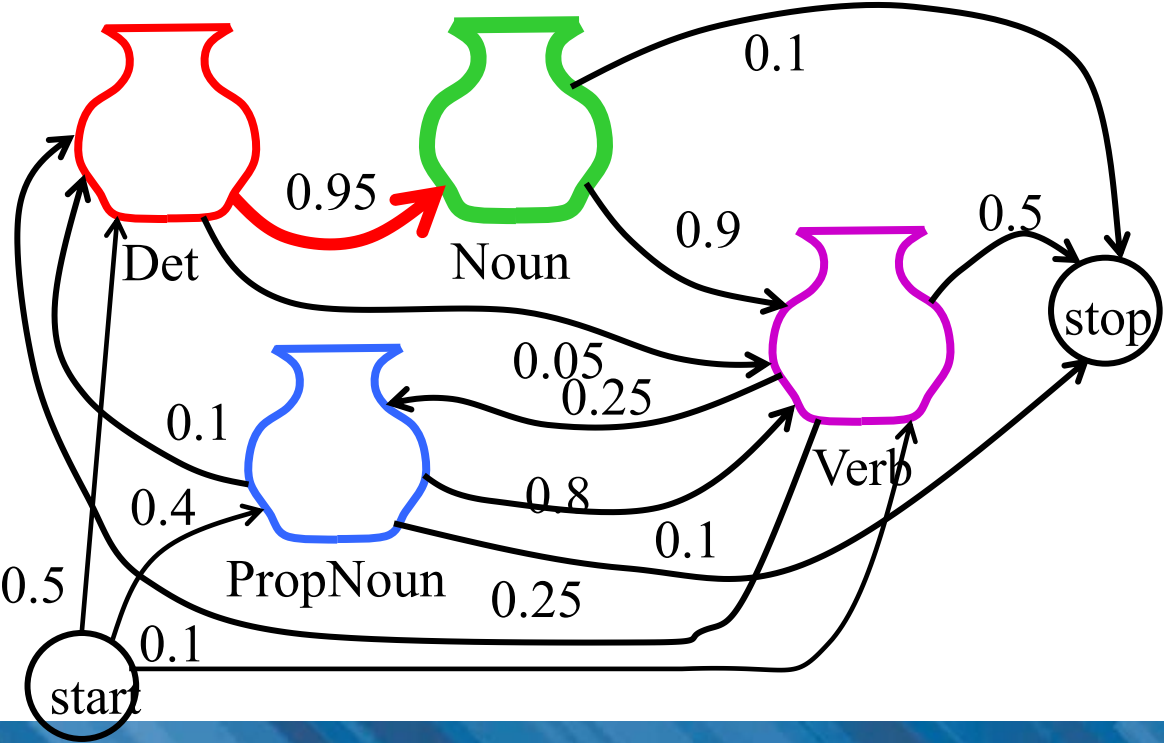
John bit the apple

	John	bit	the	apple
DET	0.01	0.04	0.95	0
Noun	0.2	0.05	0.05	0.7
PropNoun	0.75	0.05	0	0.2
Verb	0	0.9	0.05	0.05

Computing the Forward Probabilities

- Recursion

$$\alpha_t(j) = \left[\sum_{i=1}^N \alpha_{t-1}(i) a_{ij} \right] b_j(o_t) \quad 1 \leq j \leq N, \quad 1 < t \leq T$$



John bit the apple

	John	bit	the	apple
DET	0.01	0.04	0.95	0
Noun	0.2	0.05	0.05	0.7
PropNoun	0.75	0.05	0	0.2
Verb	0	0.9	0.05	0.05

Computing the Forward Probabilities

- Initialization

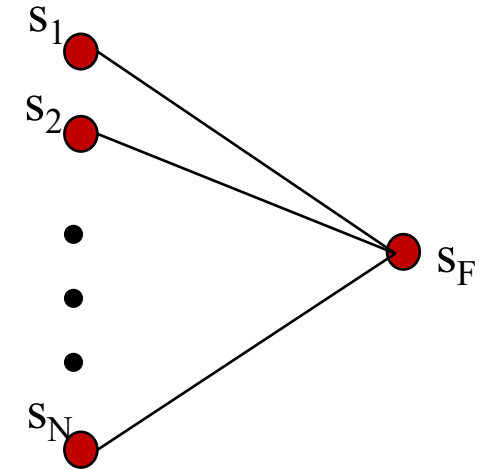
$$\alpha_1(j) = a_{0j} b_j(o_1) \quad 1 \leq j \leq N$$

- Recursion

$$\alpha_t(j) = \left[\sum_{i=1}^N \alpha_{t-1}(i) a_{ij} \right] b_j(o_t) \quad 1 \leq j \leq N, \quad 1 < t \leq T$$

- Termination

$$P(O \mid \lambda) = \alpha_{T+1}(s_F) = \sum_{i=1}^N \alpha_T(i) a_{iF}$$



Computing the Forward Probabilities

- Initialization

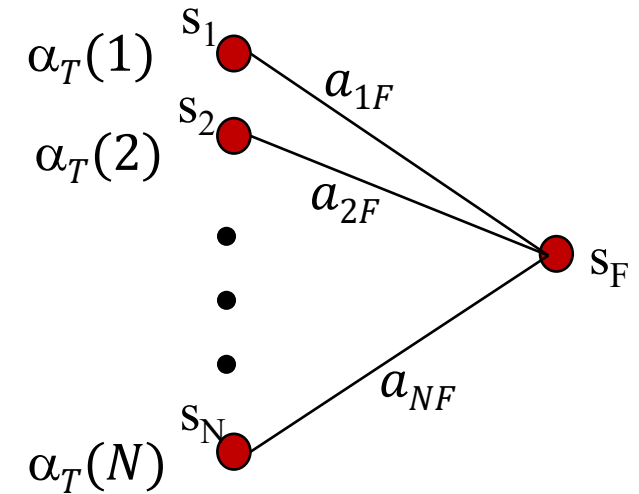
$$\alpha_1(j) = a_{0j} b_j(o_1) \quad 1 \leq j \leq N$$

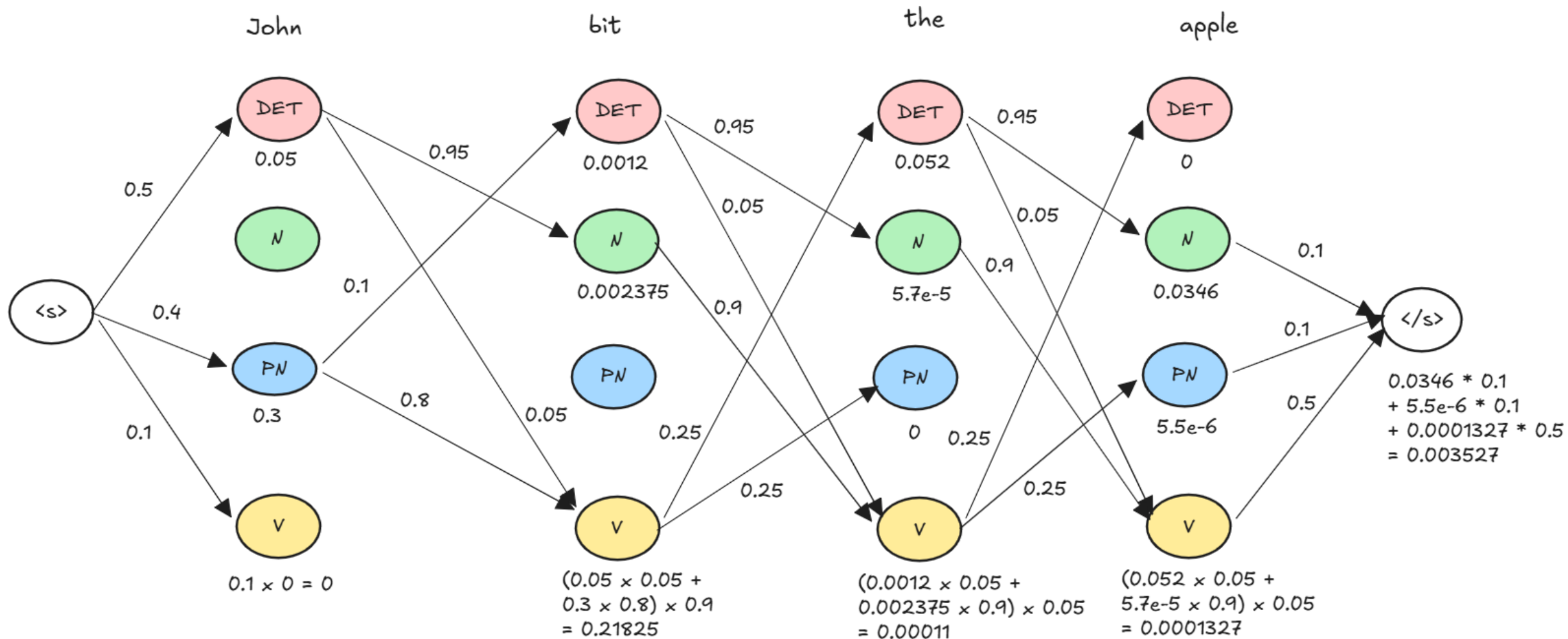
- Recursion

$$\alpha_t(j) = \left[\sum_{i=1}^N \alpha_{t-1}(i) a_{ij} \right] b_j(o_t) \quad 1 \leq j \leq N, \quad 1 < t \leq T$$

- Termination

$$P(O \mid \lambda) = \alpha_{T+1}(s_F) = \sum_{i=1}^N \alpha_T(i) a_{iF}$$





Forward Computational Complexity

- Requires only $O(TN^2)$ time to compute the probability of an observed sequence given a model (compare this with our naïve solution $O(TN^T)$).
- Exploits the fact that all state sequences must merge into one of the N possible states at any point in time and the Markov assumption that only the last state effects the next one.

Overview

1. Word categories
2. Part-of-Speech (POS) Tagging
3. Other Sequence Labelling Problems
4. Hidden Markov Model (HMM)
5. HMM: Observation Likelihood
- 6. HMM: Most Likely State Sequence**
7. HMM: Supervised Learning
8. Evaluation

Most Likely State Sequence

- Given an observation sequence, O , and a model, λ , what is the most likely state sequence, $Q=q_1, q_2, \dots, q_T$, that generated this sequence from this model?
- Used for sequence labeling, assuming each state corresponds to a tag, it determines the globally best assignment of tags to all tokens in a sequence.
- Standard procedure is called the [Viterbi algorithm](#) (Viterbi, 1967) and also has $O(TN^2)$ time complexity.

Viterbi Scores

- Recursively compute the **probability** of the **most likely subsequence** of states that accounts for the first t observations and ends in state s_j .

$$v_t(j) = \max_{q_0, q_1, \dots, q_{t-1}} P(q_0, q_1, \dots, q_{t-1}, o_1, \dots, o_t, q_t = s_j \mid \lambda)$$

- Also record “backpointers” that subsequently allow backtracking the most probable state sequence.
 - $bt_t(j)$ stores the state at time $t-1$ that maximizes the probability that system was in state s_j at time t (given the observed sequence).

Computing the Viterbi Scores

- Initialization

$$v_1(j) = a_{0j}b_j(o_1) \quad 1 \leq j \leq N$$

- Recursion

$$v_t(j) = \max_{i=1}^N v_{t-1}(i)a_{ij}b_j(o_t) \quad 1 \leq j \leq N, \quad 1 < t \leq T$$

- Termination

$$P^* = v_{T+1}(s_F) = \max_{i=1}^N v_T(i)a_{iF}$$

- Initialization

$$\alpha_1(j) = a_{0j}b_j(o_1) \quad 1 \leq j \leq N$$

- Recursion

$$\alpha_t(j) = \left[\sum_{i=1}^N \alpha_{t-1}(i)a_{ij} \right] b_j(o_t) \quad 1 \leq j \leq N, \quad 1 < t \leq T$$

- Termination

$$P(O|\lambda) = \alpha_{T+1}(s_F) = \sum_{i=1}^N \alpha_T(i)a_{iF}$$

Almost identical to the Forward algorithm except we take **max** instead of **sum**

Computing the Viterbi Backpointers

- Initialization

$$bt_1(j) = s_0 \quad 1 \leq j \leq N$$

- Recursion

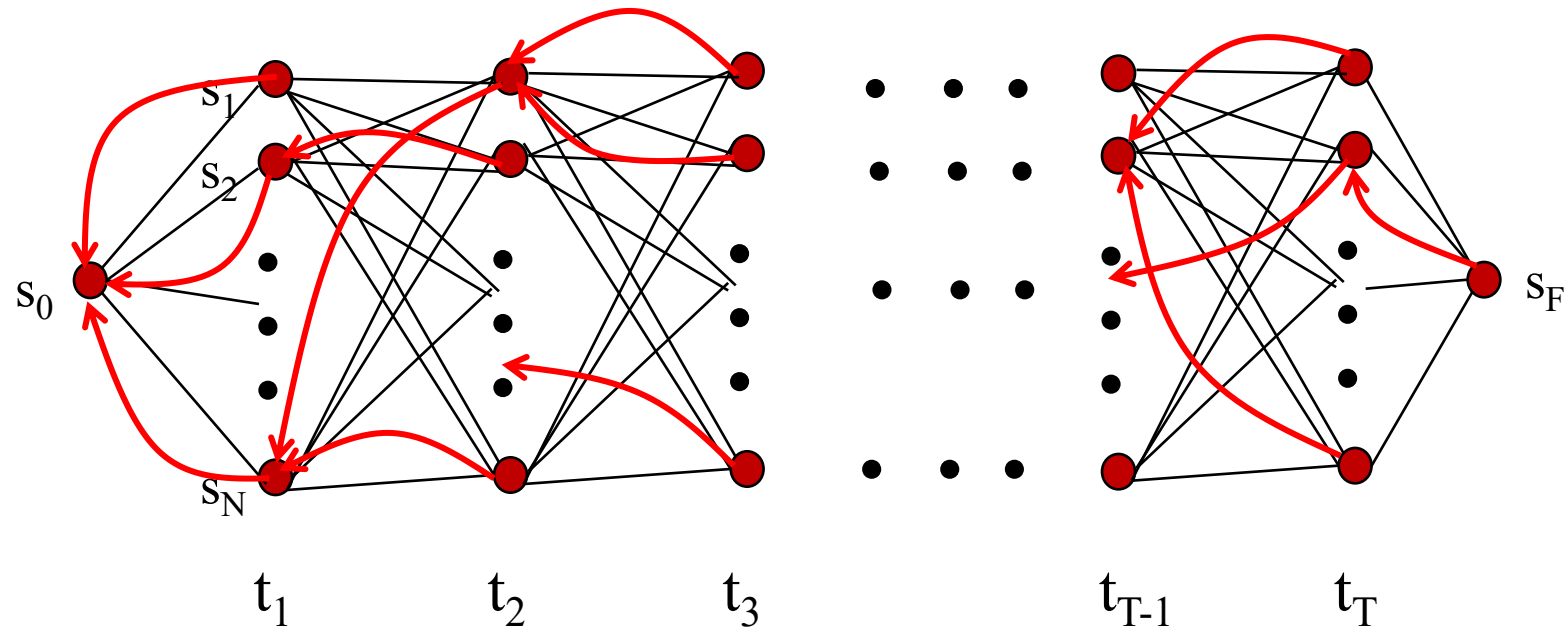
$$bt_t(j) = \operatorname{argmax}_{i=1}^N v_{t-1}(i) a_{ij} b_j(o_t) \quad 1 \leq j \leq N, \quad 1 \leq t \leq T$$

- Termination

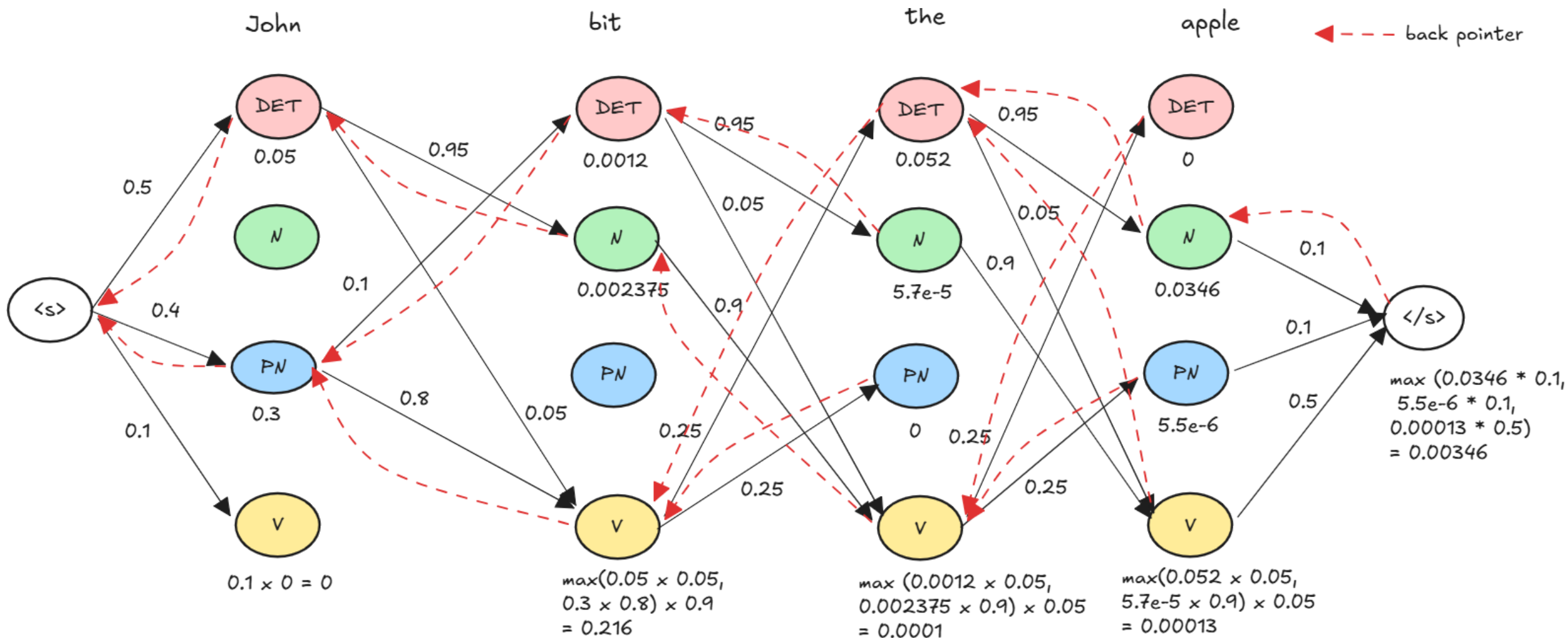
$$q_T^* = bt_{T+1}(s_F) = \operatorname{argmax}_{i=1}^N v_T(i) a_{iF}$$

Final state in the most probable state sequence. Follow backpointers to initial state to construct full sequence.

Viterbi Backpointers



Viterbi Backtrace



Overview

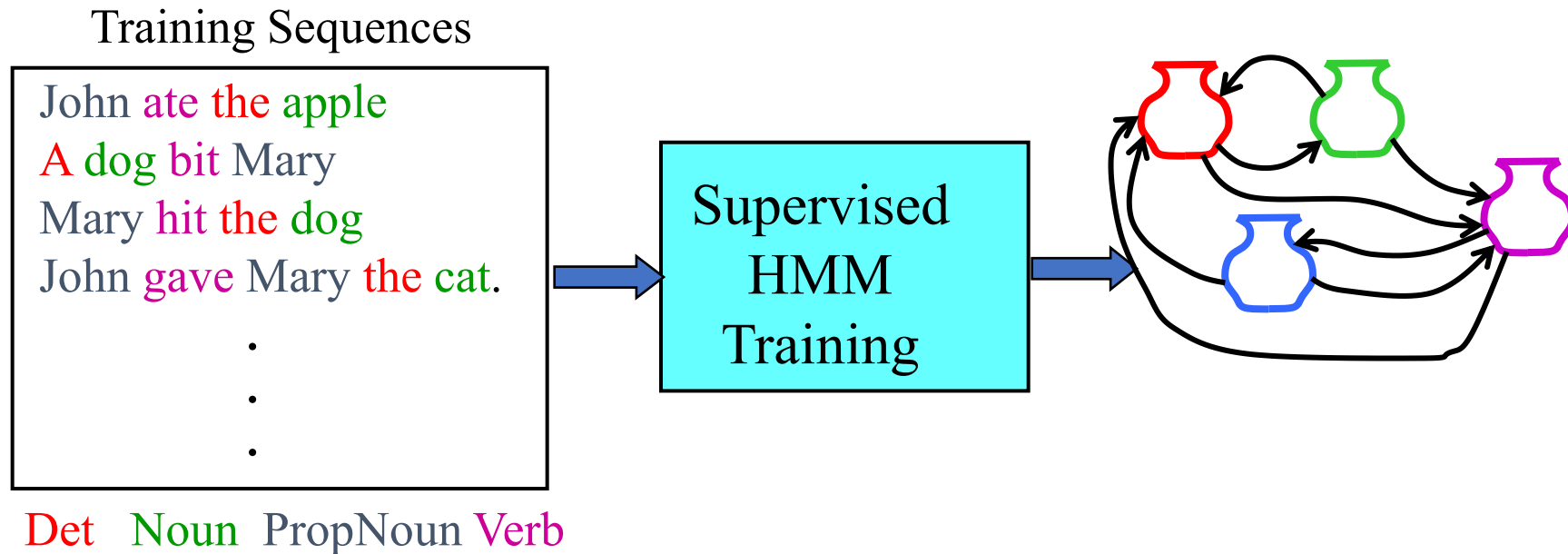
1. Word categories
2. Part-of-Speech (POS) Tagging
3. Other Sequence Labelling Problems
4. Hidden Markov Model (HMM)
5. HMM: Observation Likelihood
6. HMM: Most Likely State Sequence
7. **HMM: Supervised Learning**
8. Evaluation

HMM Learning

- **Supervised Learning:** All training sequences are completely labeled (tagged).
We only cover this.
- **Unsupervised Learning:** All training sequences are unlabeled (but generally know the number of tags/states).
- **Semi-supervised Learning:** Some training sequences are labeled, most are unlabeled.
 - We can use a method called Expectation Maximization (EM) for both unsupervised and semi-supervised learning of HMMs. EM is an iterative method for learning probabilistic categorization model from unsupervised data.

Supervised Parameter Estimation

- If training sequences are labeled (tagged) with the underlying state/POS sequences, then the parameters, $\lambda = \{A, B\}$, can all be estimated directly.
- Estimate state transition probabilities based on tag bigram and unigram statistics in the labeled data.



Supervised Parameter Estimation - continued

- Use a corpus of labeled sequence data to easily construct an HMM using supervised training.
- Given a novel unlabeled test sequence to tag, use the Viterbi algorithm to predict the most likely (globally optimal) tag sequence.

$$a_{ij} = \frac{C(q_t = s_i, q_{t+1} = s_j)}{C(q_t = s_i)}$$

- Estimate the observation probabilities based on tag/word co-occurrence statistics in the labeled data.

$$b_j(k) = \frac{C(q_i = s_j, o_i = v_k)}{C(q_i = s_j)}$$

Use appropriate smoothing if training data is sparse.

Overview

1. Word categories
2. Part-of-Speech (POS) Tagging
3. Other Sequence Labelling Problems
4. Hidden Markov Model (HMM)
5. HMM: Observation Likelihood
6. HMM: Most Likely State Sequence
7. HMM: Supervised Learning
- 8. Evaluation**

Evaluating POS Tagger performance

- The metric most often reported for POS tagging is accuracy: The proportion of words in the test corpus that were tagged correctly by the POS tagger:

$$\text{Accuracy} = \frac{\text{Number of Correct Tags}}{\text{Total Number of Tags}}$$

- Given that many words in the testing corpus have been seen during training, it makes sense to also report accuracy specifically for out of vocabulary words.
- The state-of-the-art accuracy for all words in English is almost 98%.

Feedback time!



** Please submit weekly Feedback! **

